

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

BELAGAVI-590 018



A PROJECT REPORT ON

**WKAI - WORKSHOP AI: A REAL-TIME AI-
ASSISTED WORKSHOP PLATFORM**

Submitted in partial fulfilment of the requirements for the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING

Submitted By

MUSTAFA MUHAMMAD ABDULLA 4PA22CS092

MOHAMMED SHAMMAS I 4PA23CS083

MOHAMMED ZAHID IBRAHIM 4PA23CS085

RAFAN AHAMAD SHEIK 4PA23CS102

Under the Guidance of

Prof. HABEEB UR REHMAN

Assistant Professor

Department of Computer Science & Engineering



P.A. COLLEGE OF ENGINEERING

**Affiliated to Visvesvaraya Technological University, Recognized by AICTE and Accredited by
NBA, Near Mangalore University, MANGALORE – 574 153, KARNATAKA,**

2026-27

P. A. COLLEGE OF ENGINEERING

**Affiliated to Visvesvaraya Technological University, Approved by AICTE and Accredited
by NBA, Near Mangalore University, Mangalore -574 153.**

Department of Computer Science and Engineering



CERTIFICATE

Certified that the project work entitled WKAI - **WORKSHOP AI: A REAL-TIME AI-ASSISTED WORKSHOP PLATFORM** carried out by **MUSTAFA MUHAMMAD ABDULLA - USN 4PA22CS092, MOHAMMED SHAMMAS I - USN 4PA23CS083, MOHAMMED ZAHID IBRAHIM - USN 4PA23CS085 and RAFAN AHAMAD SHEIK - USN 4PA23CS102**, bonafide students of P.A. College of Engineering in partial fulfilment for the award of bachelor of engineering in Department of Computer Science & Engineering of the Visvesvaraya Technological University, Belagavi, during the year 2026-27. It is certified that all corrections/suggestions indicated for Internal Assessment have been incorporated in the Report deposited in the departmental library. The project report has been approved as it satisfies the academic requirements in respect of Project work prescribed for the said Degree.

Signature of Guide
Prof. Habeeb Ur Rehman

Signature of the H.O.D.
Dr. Sharmila Kumari M

Signature of the Principal
Dr. Ramis M. K.

EXTERNAL VIVA

Name of the Examiners

Signature with date

1. _____

2. _____

P. A. COLLEGE OF ENGINEERING

**Affiliated to Visvesvaraya Technological University, Approved by AICTE and
Accredited by NBA, Near Mangalore University, Mangalore -574 153.**

Department of Computer Science and Engineering



DECLARATION

We, students of Seventh Semester, B.E, in the Department of Computer Science and Engineering, P.A. College of Engineering, Mangalore declare that the project entitled **WKAI - WORKSHOP AI: A REAL-TIME AI-ASSISTED WORKSHOP PLATFORM** has been carried out by us and submitted in partial fulfilment of the course requirements for the award of degree in **Bachelor of Engineering in Computer Science and Engineering** of **Visvesvaraya Technological University, Belagavi** during the academic year 2026-27. The matter embodied in this report submitted has not been presented to any other University (or) Institution for the award of any Degree (or) Diploma.

**MUSTAFA MUHAMMAD ABDULLA
MOHAMMED SHAMMAS I
MOHAMMED ZAHID IBRAHIM
RAFAN AHAMAD SHEIK**

**4PA22CS092
4PA23CS083
4PA23CS085
4PA23CS102**

Department of Computer Science and Engineering

P.A. College of Engineering

Mangalore -574153

Date: 30-11-2026

Place: Mangalore

ABSTRACT

WKAI (Workshop AI) is a real-time, AI-assisted platform for running live coding workshops and classroom sessions. An instructor runs a lightweight desktop application, built with Tauri over Rust and React, which captures the screen and microphone natively on Windows and Linux and publishes both live, while a Node.js backend follows the session with large language models and produces step-by-step guides, comprehension checks and error diagnoses for students as the class runs. Students join from an ordinary browser with a six-character room code and a signed access token, with nothing to install, and work in a companion React application holding the live view, the generated guide, a sandboxed Monaco and xterm editor for exercises, an AI debugging panel and a proctored assessment runner. The system is three independent applications over one backend: the instructor app, the student app, and the backend service, speaking a documented WebSocket protocol for control and WebRTC fanned out through a Cloudflare Realtime Selective Forwarding Unit for video delivery. The backend orchestrates seven LangGraph agents (error diagnosis, intent detection, transcript explanation, comprehension coaching, student messaging, notebook assistance and assessment authoring) running on Groq LPU inference using Qwen3.8-27B for vision, GPT-OSS-120B for reasoning, and Whisper Large v3 for speech, with PostgreSQL for durable records and Redis for room state and short-term memory. Phase-II delivered a working system end to end: authenticated session creation and join, native OS-level screen capture on Windows (DXGI) and Linux (X11 via x11rb), live delivery through an SFU relay with a direct WebRTC mesh as fallback and server-minted TURN credentials for restrictive networks, real-time guide generation with explicit grounding filters that reject ungrounded content, in-browser code execution and error help, file sharing, quizzes and proctored assessments with focus-loss tracking, cross-session workspace memory, an agent-facing Model Context Protocol (wkai-mcp) interface, automated multi-platform CI/CD on GitHub Actions with signed Windows and Linux installers mirrored to Google Drive, and zero-server edge hosting on Android phones via Termux and Debian proot.

ACKNOWLEDGEMENT

The successful completion of any task would be incomplete without mentioning the people who made it possible. So, it is with gratitude that I acknowledge the help, which crowned our efforts with success.

We hereby thank our project guide **Prof. Habeeb Ur Rehman**, Department of Computer Science & Engineering, P. A. College of Engineering, Mangalore, for his guidance, advice, inspiration and unconditional support which helped us complete our project successfully.

We thank our coordinator **Dr. Hafeez M K** and **Mrs. Sakeena** Department of Computer Science & Engineering, P. A. College of Engineering, Mangalore, for their valuable guidelines.

We thank our HOD **Dr. Sharmila Kumari M**, Department of Computer Science & Engineering, P. A. College of Engineering, Mangalore, for being a source of inspiration and being a helping hand whenever required.

We would also like to thank **Dr. Ramis M. K**, Principal, P. A. College of Engineering, Mangalore, for his immense support throughout the course of the project.

We are also grateful to the Department of **Computer Science and Engineering** and our institution **P.A. College of Engineering** for imparting us the knowledge with which we could do our best. Also, we would like to thank the whole teaching and non- teaching staff of Computer Science and Engineering department for their help and support rendered throughout the course of the project.

TABLE OF CONTENTS

INTRODUCTION	1
1.1 Background	1
1.2 Motivation	2
1.3 Overview of the Proposed System	3
1.4 Summary of Contributions	4
1.5 Intended Users and Deployment Context	5
1.6 Literature Survey	5
1.6.1 Domain Overview	5
1.6.2 Review of Related Work	6
1.6.3 Comparative Positioning of Existing Tool Categories	9
1.6.4 Research Gap Identified	10
1.7 Problem Statement	11
1.8 Objectives	11
1.9 Scope of the Project	12
1.10 Organization of the Report	12
SOFTWARE REQUIREMENTS SPECIFICATION	13
2.1 Functional Requirements	13
2.2 Non-Functional Requirements	13
2.2.1 Prioritization of Non-Functional Requirements	14
2.3 Assumptions and Constraints	14
2.4 Feasibility Study	15
2.4.1 Technical Feasibility	15
2.4.2 Economic Feasibility	15
2.4.3 Operational Feasibility	15
2.5 Software Requirements	15
2.5.1 Prerequisites for Development	16
2.6 Minimum System Configuration	17
2.7 External Interface Requirements	17
2.7.1 User Interfaces	17
2.7.2 Software Interfaces	17
2.7.3 Communication Interfaces	17
2.8 Requirements Traceability Matrix	18
2.9 Expected Outcomes	18
HIGH-LEVEL DESIGN	20
3.1 System Architecture	20
3.2 Software Design	21

3.2.1 Persistent Data Model	23
3.2.2 Ephemeral State Model	24
3.2.3 Module Structure of the Backend	24
3.2.4 Client Application Structure	25
3.3 Interface between Components (Protocol).....	25
3.4 Design Patterns Applied.....	28
3.5 Design Trade-offs and Rationale	29
DETAILED DESIGN	30
4.1 AI Agent Layer Design	30
4.1.1 Common Agent Contract	30
4.1.2 Error Diagnosis Agent	30
4.1.3 Intent Detection Agent	31
4.1.4 Transcript Explanation Agent	31
4.1.5 Comprehension Coaching Agent	31
4.1.6 Student Messaging Agent	32
4.1.7 Notebook Assistance Agent	32
4.1.8 Resource Access Agent.....	32
4.1.9 Concurrency and Rate Control.....	33
4.1.10 Structured Output Validation	33
4.2 Security Architecture	34
4.2.1 Token-Based Identity	34
4.2.2 Authorization Boundaries	34
4.2.3 Supporting Controls	34
4.2.4 Residual Risks.....	35
IMPLEMENTATION.....	36
5.1 Development Environment Setup	36
5.2 Module-wise Implementation	36
5.2.1 Student Exercise Sandbox.....	37
5.2.2 File Sharing Pipeline	38
5.2.3 Guide Delivery and Reconnection	38
5.2.4 Instructor Presence Signalling	38
5.3 Build, Release, and Deployment Automation.....	38
5.4 Implementation Challenges Encountered	39
5.5 Screenshots of the Delivered System	40
TESTING AND RESULTS.....	41
6.1 Testing Strategy	41
6.2 Test Cases and Results	41
6.3 Unit Test Coverage	42

6.3.1 Unit Test Cases	43
6.4 Performance Observations	46
6.5 Security Testing	47
6.6 Usability Verification.....	47
6.7 Results Overview	47
6.8 Objective-wise Evaluation	49
6.9 Discussion	49
CONCLUSION AND FUTURE SCOPE	51
7.1 Conclusion	51
7.2 Limitations	51
7.3 Future Scope	51
7.3.1 Near-Term Work.....	52
7.3.2 Medium-Term Work	52
7.3.3 Longer-Term Directions	52
7.4 Concluding Remarks.....	53
REFERENCES	54
APPENDIX A: SOURCE CODE	55
A.1 Repository Layout	55
A.2 Key Modules	55
A.3 Build and Run Instructions.....	55
APPENDIX C: PLAGIARISM REPORT	55
APPENDIX D: JOURNAL / CONFERENCE CERTIFICATE	56
APPENDIX E: BIOGRAPHY (STUDENT & GUIDE)	56
F.1 Project Management Approach	57
F.2 Work Breakdown Structure (WBS).....	57
F.3 Task-wise Time Schedule (Gantt Chart Data).....	57
F.4 Software Project Management.....	58
F.4.1 Development Methodology	58
F.4.2 Version Control & Configuration Management	58
F.4.3 Roles and Responsibilities	59
F.4.4 Software Cost Estimation	59
F.6 Risk Management (Software)	60
F.9 Cost of Operation.....	60

LIST OF FIGURES

- Figure 3.1 – WKAI system architecture: the three applications, the relay, and the backing services
- Figure 3.2 – Capture-to-guide AI pipeline, including the two gates that reject ungrounded output
- Figure 3.3 – Session message sequence: opening a room, a student arriving, teaching, and getting unstuck
- Figure 5.1 – Student web application: landing page
- Figure 5.2 – Joining a session with a room code
- Figure 5.3 – Student live guide, written from the screen and the instructor's speech
- Figure 5.4 – Files shared by the instructor
- Figure 5.5 – AI Helper: sandbox, error helper and notebook assistant
- Figure 5.6 – Private question thread, answered by the instructor
- Figure 5.7 – Instructor application: a live session with the present bar and side rail
- Figure 5.8 – Instructor application: settings, appearance and AI controls

LIST OF TABLES

- Table 1.1 – Summary of Literature Survey
- Table 2.1 – Software Stack / Tooling
- Table 2.2 – Requirements Traceability Matrix
- Table 3.1 – Persistent (PostgreSQL) Data Model
- Table 3.2 – Key WebSocket Message Types
- Table 3.3 – Key Backend REST Routes
- Table 4.1 – AI Agent Layer: Triggers, Contracts, and Failure Behaviour
- Table 6.1 – Test Cases and Results
- Table 6.2 – Observed Latency Characteristics
- Table 6.3 – Evaluation Against Phase-II Objectives
- Table 6.4 to 6.10 – Unit test cases (UTC 1.1 to UTC 1.7)
- Table F.1 – Task-wise Time Schedule
- Table F.2 – Recurring Cost Structure and Scale Drivers

LIST OF ABBREVIATIONS

Abbreviation	Expansion
SRS	SOFTWARE REQUIREMENTS SPECIFICATION
WBS	Work Breakdown Structure
SDLC	Software Development Life Cycle
IDE	Integrated Development Environment
API	Application Programming Interface
LLM	Large Language Model
WS	WebSocket
SFU	Selective Forwarding Unit
WHIP	WebRTC-HTTP Ingestion Protocol
RTMP	Real-Time Messaging Protocol
TTL	Time To Live

Chapter 1

INTRODUCTION

1.1 Background

Live coding workshops and classroom programming sessions are usually run on generic screen-sharing tools such as Zoom or Google Meet, which have no awareness of what is being taught, or on LMS platforms that were never built for real-time, hands-on instruction. Neither kind of tool understands the content of a session. A student who loses attention for thirty seconds, or who hits an error, has no way to catch up except by interrupting the instructor. Meanwhile large language models hosted on fast inference providers such as Groq [11] have become cheap and quick enough to process a live transcript and screen stream at close to real time, which makes an AI co-instructor practical: one that follows the session alongside the students and helps them directly.

The problem is structural, not incidental. A video-conferencing tool is designed to move pixels and audio samples, and it deliberately treats session content as opaque payload. That is the right decision for a general meeting product, but it means everything of teaching value produced during a live session (the commands the instructor typed, the reasoning they explained aloud, the error they debugged in front of the class) is transmitted once, rendered once, and then discarded. Nothing is indexed, nothing is summarised, and nothing is available to a student who lost the thread five minutes ago. A Learning Management System sits at the other extreme. It is built entirely around persistent artefacts, but those artefacts are written ahead of time by the instructor as separate manual work. Neither closes the loop between what happens live and what remains afterwards.

A second gap concerns feedback direction. In a physical workshop the instructor reads the room: puzzled faces, students who have stopped typing, several hands going up at the same step. Remote delivery removes nearly all of that. What is left is a grid of muted thumbnails and a chat window that only the most confident students use. The instructor carries on assuming a level of comprehension that often is not there, and the students who fall behind do so invisibly. Any tool meant to improve remote technical instruction has to rebuild that feedback channel from data it can actually observe: the words being spoken, the content on screen, the commands students run, and the errors those commands produce.

The economics of inference have also changed, which is what makes the problem tractable now. Until recently, several language-model calls per minute of a live class were too slow and

too expensive to consider. A classification call plus a generation call could take tens of seconds, well outside what a live session tolerates. Purpose-built inference hosting, in particular the LPU-based service offered by Groq [11], has brought typical latency for mid-sized models down to fractions of a second, and speech transcription to well under real time. That moves an AI co-instructor out of the research-demonstration category and into ordinary engineering: the models are fast enough, and the remaining difficulty is capture, orchestration, state management and delivery. WKAI is an attempt to solve that engineering problem end to end.

1.2 Motivation

WKAI started from a gap in the tools instructors already use. Someone running a live coding session has no automated way to produce a written guide of what was taught, no way to tell which students are quietly stuck, and no cheap way to give every student individual debugging help. Students have no record of the session beyond their own notes, and no way to get unstuck without breaking the instructor's flow. The goal is to let the instructor teach normally, screen and voice and nothing else, while the system builds the guide, watches for confusion, and gives each student a private assistant grounded in what is being taught at that moment.

The second motivation was noticing that the three things instructors most often want after a session (a written record, a comprehension check, and individual debugging support) can all be derived from data the session already produces. The written record is a summarisation problem over the transcript and the screen. The comprehension check is a question-generation problem over the same material. Debugging support is a diagnosis problem over the student's error text, grounded in what was just taught. None of the three asks the instructor to do more work. All three only require that something is listening. Treating the project as extracting value from a signal that already exists, rather than adding another instructor obligation, became its central design principle, and it is why the instructor-facing surface was kept to one form and one button.

The team's own experience in laboratory and workshop courses supplied the last motivation. In a two-hour practical session a large share of instructor time goes on repeated, individually trivial interventions: an environment variable that was never exported, a dependency installed into the wrong interpreter, a typo in a path. Those interruptions cost twice over. They stall the student who asked, and they fragment the session for everyone else. An assistant that clears the long tail of routine errors on its own, while passing genuine conceptual confusion to the instructor, would return a good part of that teaching time without diluting the instructor's role.

1.3 Overview of the Proposed System

WKAI is software only, structured across three independent components. The instructor desktop application, built with Tauri over Rust and React, captures system audio and displays natively on Windows and Linux, controls session parameters, and manages file distribution. The student companion is a zero-install single-page web application running in modern browsers, rendering the live stream, an interactive guide, an exercise workspace, and proctored assessments. The backend, implemented in Node.js with TypeScript and Express, manages rooms, verifies signed tokens, routes WebSocket messages, and orchestrates the AI agent pipeline on Groq.

The three applications are deliberately decoupled. They share no code beyond a common vocabulary of message types, they talk only over documented WebSocket and REST contracts, and each can be developed, deployed and versioned on its own. The split exists so that the instructor application, the only part that needs a native toolchain and per-operating-system capture code, can move at its own pace without holding up the browser-only student application, and so that the backend can be redeployed without students reloading or reinstalling anything. The integration surface is the protocol contract rather than a shared library, and Chapter 3 documents it in full.

Key Features Delivered

- Zero-effort guide generation: the instructor teaches normally, WKAI watches the screen and listens, and writes the guide.
- Grounded output: blocks that only narrate the screen, or that carry no code where code is claimed, are rejected before a student ever sees them.
- Comprehension gating: the guide stays locked until the student answers the check correctly.
- Error diagnosis: paste any terminal error and get a cause in plain language plus a command to run, without interrupting the class.
- Native capture: screen and microphone inputs are grabbed directly through operating system APIs (Windows Desktop Duplication API with GDI fallback; Linux X11 protocol via x11rb with PipeWire fallback), avoiding browser permission dialogs and webview capture overhead.
- Relay delivery: the instructor publishes one media stream to a selective forwarding unit (SFU) and every student pulls a copy, so the instructor's upstream bandwidth and CPU remain constant as the class grows.

- Direct mesh fallback: a room with no relay configured, or one whose pull fails, drops back to peer connections and keeps working.
- Server-minted TURN credentials: short-lived relay credentials are issued by the backend, so a campus network that blocks peer traffic does not end the session.
- Sandboxed code execution: Python, JavaScript, TypeScript or Bash, run from the browser.
- Assessments and proctoring: timed comprehension quizzes and programming challenges with client focus-loss ('away modal') and paste-event recording for instructor review.
- File sharing: the instructor shares from a watched folder, an upload or a URL, and students download instantly.
- Intent-aware audio: the AI hears “share this file” in speech and offers it to the instructor rather than acting on its own.
- Workspace memory: sessions in the same workspace share what was taught earlier in the course.
- Session recording: the instructor can keep a local copy of the session on disk.
- An agent interface for tooling: an MCP server (wkai-mcp) exposes eight tools (wkai_create_session, wkai_connect_instructor, wkai_send_screen_frame, wkai_speak, wkai_share_file, wkai_reply_to_student, wkai_list_students, wkai_end_session) letting an external agent run a complete workshop session.
- Centralised AI agents: every agent shares one lifecycle contract (name, version, invoke, health check) so a new capability is a file rather than a refactor.
- System tray operation: the desktop app stays out of the way while the class runs.

1.4 Summary of Contributions

The contributions of this phase can be stated precisely. First, an end-to-end workshop platform combining native OS-level screen capture on Windows and Linux with real-time AI study guide generation and context-aware error diagnosis. Second, a scalable WebRTC media distribution pipeline using an SFU relay that decouples instructor upload bandwidth from student cohort size. Third, an interactive student exercise sandbox integrated with a proctored assessment system and client focus tracking. Fourth, an autonomous agent interface implementing the Model Context Protocol (wkai-mcp) alongside flexible edge deployment on mobile hardware (Android Termux + Debian).

The architecture preserves clear boundaries between capture, distribution, and inference. Rather than binding screen capture to fragile browser extensions, the instructor client captures

framebuffers through native Rust subsystems (DXGI on Windows, x11rb on Linux). Video transmission uses an SFU relay with direct peer mesh fallback. Authentication relies on server-minted JSON Web Tokens, ensuring roles and capabilities cannot be spoofed by client-side manipulation.

1.5 Intended Users and Deployment Context

WKAI targets three groups. The primary user is an instructor running a live, hands-on technical session for five to about sixty students: a laboratory batch, a departmental workshop, a bootcamp cohort, a corporate training group. That instructor is assumed to be technically capable but short of time, unwilling to learn a new production workflow, and unwilling to slow the session down in order to operate tooling. The second group is the students, joining from a personal or laboratory machine over a network of variable quality, possibly without administrative rights, who therefore must not have to install anything. The third is the department or institution, which cares about session records, attendance evidence, and controlled access to live sessions.

The deployment context follows from those assumptions. The backend runs as a managed cloud service with hosted PostgreSQL and Redis instances, the student application is served as static assets from a content-delivery network, and the instructor application ships as a signed desktop installer built automatically from the main branch. Nothing requires on-premises server administration, which is deliberate: the people operating it are teaching staff, not infrastructure engineers. Session records outlive the live session so that guides and error logs stay retrievable, while ephemeral state such as room membership and short-term conversational memory sits in Redis with a bounded time to live, so abandoned sessions clear themselves.

1.6 Literature Survey

1.6.1 Domain Overview

WKAI sits between three existing categories of tool. Generic video-conferencing and screen-share software (Zoom, Google Meet, Microsoft Teams) gives reliable live video but no understanding of session content, so it cannot produce guides, quizzes or targeted help. Learning Management Systems (Moodle, Google Classroom) handle static course content and assignments well, but they are not built for a live, in-session experience. AI coding assistants (GitHub Copilot and chat tools of that kind) are strong at one-to-one code help, yet they have no notion of what is being taught right now in this class; they see only what the student pastes

in. On the delivery side, SFU infrastructure such as LiveKit [10] and open WebRTC ingestion standards such as WHIP [14] now let a small team build one-to-many low-latency streaming without running a custom media server, which is the approach this project takes. Groq's LPU-based hosting was chosen because it makes several model calls per message (classification, diagnosis, guide generation) fast enough to feel immediate [11], which slower hosted providers do not.

1.6.2 Review of Related Work

Nine works were reviewed in detail. Each was chosen because it addresses one of the four capabilities WKAI has to combine: tutoring dialogue, real-time collaboration between a human and a model, screen and lecture understanding, and programming-specific error assistance. Together they set out the state of the art and the gap this project addresses.

The comprehensive review of AI-based intelligent tutoring systems [1] gives the taxonomic backdrop for the whole project. It classifies tutoring systems by where the intelligence sits, in the domain model, the student model or the pedagogical model, and observes that language-model systems have largely collapsed the first two into a single prompted context window, trading interpretability for breadth. Its most relevant finding for WKAI is that the systems reporting the strongest learning outcomes are the ones that ground their responses in a specific, bounded body of course material rather than in general world knowledge. That is why WKAI builds its agent prompts from the live session transcript and the current guide state instead of letting the model answer from pre-training alone.

Ruffle & Riley [2] reports on a conversational tutoring system built on a language model, in which two agents take distinct pedagogical personas. Its contribution here is methodological rather than architectural. The paper documents that unconstrained conversational tutors tend to volunteer answers, cutting short the productive struggle that produces learning. The authors' fix, an explicit instruction hierarchy in which the tutor must elicit before it explains, shaped WKAI's comprehension-coaching agent, which gates progression on a correct student answer instead of showing the answer next to the question.

Tutor CoPilot [3] is the closest conceptual match to WKAI's operating model among the reviewed works. Instead of replacing the human tutor, it augments a live session by suggesting expert moves in real time. Its central empirical result, that the gains were largest for the least experienced tutors and for the students furthest behind, supports WKAI's position as an instructor amplifier rather than an instructor substitute. The latency constraints it reports also

confirmed that WKAI's assistance has to arrive within a few seconds to be usable during instruction rather than after it.

ScreenAI [4] presents a vision-language model specialised for user-interface screenshots and infographics, covering layout parsing, widget identification and question answering over screen content. This is the technical foundation for the screen-understanding half of WKAI's ingestion pipeline. A spoken transcript on its own is not enough context, because much of the instructional signal in a coding session is visual: the file being edited, the command in the terminal, the stack trace on screen. ScreenAI shows that this content is machine-interpretable at usable accuracy, which is what justifies treating captured frames as a first-class input to guide generation rather than as opaque video.

MultiTutor [5] proposes a collaborative multi-agent architecture for multimodal student support, where specialised agents handle distinct support modes and coordinate through shared state. The parallel to WKAI's centralised agent layer is direct, but the paper's warning is more useful: as the number of agents grows, undisciplined communication between them becomes the main source of latency and failure. WKAI answers that with a common agent contract carrying explicit name, version, invocation and health-check members, and a rule that agents communicate only through the LangGraph state object [12] rather than calling one another.

MMTutorBench [6] introduces the first multimodal benchmark for AI mathematics tutoring, and defines evaluation dimensions that separate correctness of the answer from the quality of the pedagogical intervention. WKAI's domain is programming rather than mathematics, but that separation was adopted directly in this project's evaluation: a diagnosis that finds the right root cause and then presents it as an unexplained command is scored a partial success, because the student learns nothing transferable from it.

Video-MMLU [7] contributes a large multi-discipline benchmark for lecture-video understanding, testing whether a model can reconstruct the conceptual structure of a recorded lecture from its audio and visual content. That is the task WKAI's transcript-explanation agent performs, except that WKAI has to do it incrementally on a streaming input rather than retrospectively on a finished recording. The benchmark's finding that segment-level summarisation substantially outperforms whole-lecture summarisation is why WKAI emits the guide as a sequence of discrete, individually timestamped blocks instead of one continuously rewritten document.

MathBuddy [8] demonstrates an affective tutoring system that infers a student's emotional state from multimodal signals and adapts its intervention. WKAI does not implement affect

detection, and the paper is included partly to mark that boundary. Its useful indirect contribution is the demonstration that low-cost proxies for engagement, such as response latency, retry counts and abandonment, carry usable signal without any camera access. That is how WKAI infers a student is stuck: repeated failed executions and unanswered comprehension questions, not facial analysis.

LeafTutor [9] is the closest prior work to WKAI's error-diagnosis agent. It tutors students through programming assignments by reasoning over their submitted code and its failures, and its two-stage pattern, classify the failure then generate a targeted intervention, is the one WKAI adopts in its error graph. The decisive difference is context. LeafTutor addresses an asynchronous assignment with a known specification and a known test suite, while WKAI has to diagnose an error produced in a live session with no specification, no test suite, and only the live transcript as a statement of intent. That constraint is why WKAI's diagnosis prompt is built from session context at invocation time rather than from a fixed problem definition.

Table 1.1 – Summary of Literature Survey

Sl.No	Author(s) / Title	Year	Venue	Relevance / Findings
1	LeafTutor: An AI Agent for Programming Assignment Tutoring	2026	ArXiv	Closest prior work to WKAI's programming-specific error-diagnosis agent
2	A Comprehensive Review of AI-based Intelligent Tutoring Systems: Applications and Challenges	2025	ArXiv	Surveys the AI-tutoring landscape WKAI's guide/error-diagnosis pipeline sits in
3	MultiTutor: Collaborative LLM Agents for Multimodal Student Support	2025	AI-Supported Education (PMLR)	Multi-agent tutoring architecture; parallels WKAI's centralized multi-agent layer
4	MMTutorBench: The First Multimodal Benchmark for AI Math Tutoring	2025	ArXiv	Benchmarking approach for multimodal tutoring quality
5	Video-MMLU: A Massive Multi-Discipline Lecture Understanding Benchmark	2025	ArXiv / CVPR	Lecture-understanding benchmark relevant to transcript/guide generation
6	MathBuddy: A Multimodal System for Affective Math Tutoring	2025	EMNLP 2025 Demo	Multimodal affective tutoring system design
7	Ruffle & Riley: Insights from Designing and Evaluating an LLM-Based Conversational Tutoring System	2024	AIED 2024 Workshop	Design lessons for conversational LLM tutors; informs WKAI's message agent
8	Tutor CoPilot: A Human-AI Approach for Scaling Real-Time Expertise	2024	NSSA Technical Report (Stanford)	Human-in-the-loop real-time tutoring; close conceptual match to WKAI's live-session model
9	ScreenAI: A Vision-Language Model for UI and Infographics Understanding	2024	IJCAI 2024	Vision-model grounding for screen content; relevant to WKAI's screen-analysis pipeline

Sl.No	Author(s) / Title	Year	Venue	Relevance / Findings
9	Instructor joins a room that already has students	The roster the instructor sees names every student present	Membership changes now resend the whole list; an instructor arriving second no longer sees an empty panel beside a non-zero count	Pass
10	A student's slow database read during join	That one socket fails; the room is unaffected	The connection handler no longer rejects into an unhandled promise that ended the process	Pass
11	A room with no SFU credentials configured	The session still streams	Both clients fall back to the direct peer mesh; behaviour matches the previous phase	Pass
12	The vision model is asked for more output tokens than the plan allows	The failure is reported as a configuration limit, not retried	Over-large requests and spent daily quotas are now classified as non-retryable and named in the instructor's console	Pass
13	A guide block that only narrates the screen	It never reaches a student	Rejected by the grounding filter, with the reason reported rather than silently swallowed	Pass

WKAI differs from all of these in combining live screen and audio capture, real-time guide generation, and programming-error diagnosis in one system built around a live instructor session, rather than around an offline assignment or a purely conversational tutor.

1.6.3 Comparative Positioning of Existing Tool Categories

Setting WKAI against the three incumbent categories clarifies what is actually new. Against conferencing platforms it adds on the pedagogical axis and falls behind on the general-purpose one: it offers guide generation, comprehension gating and error diagnosis that no conferencing tool provides, and makes no attempt at breakout rooms, calendar integration, large webinars or telephony. The narrowing is deliberate. A conferencing platform has to serve every meeting shape, which is exactly why it cannot afford to model session content. WKAI serves one meeting shape and can therefore model it in depth.

Against Learning Management Systems the difference is temporal. An LMS is authoritative about material that exists before and after a session, such as syllabi, assignments, submissions and grades, and WKAI does not compete there. WKAI is authoritative about the session itself, and the artefact it produces at the end is exactly the kind of object an LMS stores well. The two complement each other, and the guide-block persistence model was designed with eventual export to an LMS in mind.

Against general AI coding assistants, the difference is grounding and initiative. A chat assistant is reactive and context-blind: it knows only what the student pastes into it, so a student who does not know what to ask gets nothing. WKAI's agents are invoked by system events, such as a failed execution, an unanswered comprehension question or a detected spoken intent, and their prompts are built from session state the student never has to describe. The practical result is that WKAI can help a student who has not yet formulated a question, which is the group that most needs help and the one reactive assistants cannot reach.

1.6.4 Research Gap Identified

The reviewed literature establishes each required capability in isolation and, in several cases, to a higher standard than this project reaches. Conversational tutoring is well studied [2]. Screen understanding is demonstrated [4]. Multi-agent coordination is characterised [5]. Programming-error tutoring is implemented [9]. What the literature does not provide is an integrated system in which these capabilities operate on a live, unscripted session in real time. Every reviewed system assumes at least one of the following: that the material is known in advance, that the interaction is asynchronous, that a specification or test suite defines correctness, or that the student starts the exchange by describing the problem.

WKAI's contribution is therefore integrative and engineering-led rather than model-led. The gap it addresses is the absence of a demonstrated architecture that ingests live screen and audio, keeps bounded session memory under real-time constraints, orchestrates several specialised agents without exceeding upstream rate limits, and delivers the results to a browser client over an authenticated real-time channel, while staying operable by one instructor with no extra workflow. The rest of this report documents the design and implementation of such an architecture, the security model that makes it deployable, and the measured behaviour of the delivered system.

1.7 Problem Statement

Instructors running live coding workshops have no tool that both streams their session and understands its content well enough to automatically produce a study guide, detect confused or stuck students, and provide each student individualized, context-aware debugging help, all without adding manual overhead to the instructor's teaching.

The statement breaks into four sub-problems, and naming them separately is useful because they fail in different ways. Capture and delivery asks how a session's screen and audio can be acquired at adequate fidelity and delivered to many concurrent viewers at low latency without the instructor operating broadcasting software; it fails as degraded or unreliable video. Comprehension asks how an unstructured, continuous stream of speech and screen content becomes a coherent guide published as it goes; it fails as a guide that is either too fragmentary to follow or too late to use during the session.

The third sub-problem is detection: working out that a particular student is stuck, with no explicit report from the student and no surveillance-style monitoring. It fails silently, which is the worst case, since a student falls behind and nothing notices. The fourth is intervention: generating a targeted, correct and pedagogically sensible response and delivering it to exactly the student who needs it, grounded in what was actually taught rather than in generic knowledge. It fails as a confidently wrong answer, which is worse than no answer because it costs the student their trust as well as their time. A fifth problem cuts across the other four. Access to a live session, and the privileges held inside it, must be provably controlled, because a session carries the instructor's voice, their screen, and the identity of every participant.

1.8 Objectives

1. Build an instructor application that captures and streams screen and audio natively on Windows and Linux with minimal setup.
2. Build a backend AI pipeline that turns the live audio transcript and screen content into a structured, continuously updated guide for students.
3. Detect student errors/confusion (via code execution errors and explicit questions) and generate targeted, context-grounded AI diagnoses and answers.
4. Deliver the live stream and guide to students through a zero-install browser application with embedded exercises and proctored comprehension assessments.
5. Harden authentication, session lifecycle, and multi-platform packaging with automated CI/CD and edge hosting support.

1.9 Scope of the Project

In scope for this phase: instructor screen and audio capture and streaming across Windows and Linux, the WebSocket signalling and session-lifecycle protocol, server-minted join tokens, the seven core LangGraph agents on Groq (Qwen3.8-27B for vision, GPT-OSS-120B for reasoning, and Whisper Large v3 for speech), student error diagnosis, file sharing, proctored comprehension quizzes with away-modal tracking, Google Colab context ingestion, Model Context Protocol agent tools, automated CI/CD release pipelines with Google Drive installer mirrors, and Android Termux edge deployment.

The scope boundary was drawn on one criterion: whether a capability could be validated end to end within the phase. Every in-scope item can be exercised by starting the three applications, joining a session from a browser and watching what happens. Each out-of-scope item depends on infrastructure or platform work whose completion could not be guaranteed inside the phase, such as a media relay deployment, a second native application, or a distributed state migration, and including them would have produced half-working features that could not be demonstrated honestly. Where an out-of-scope capability was still a prerequisite for later work, the groundwork was completed and is reported as such. The signed join-token infrastructure, for example, was built during this phase precisely because the planned student companion application will need it as a launch mechanism.

1.10 Organization of the Report

Chapter 1 introduces the problem, reviews the related literature, and states the objectives and scope of the work. Chapter 2 specifies the functional and non-functional requirements, the software requirements, the constraints, the feasibility study and the requirements traceability matrix. Chapter 3 presents the high-level design: the system architecture, the persistent and ephemeral data models, and the protocol between the three applications. Chapter 4 gives the detailed design of the AI agent layer and the security architecture. Chapter 5 describes the implementation module by module, with screenshots of the delivered system. Chapter 6 reports the testing strategy, the test results, the performance observations and the evaluation against the stated objectives. Chapter 7 concludes and sets out the limitations and future scope. The appendices carry the source-code layout, the project management record and the supporting certificates.

Chapter 2

SOFTWARE REQUIREMENTS SPECIFICATION

2.1 Functional Requirements

Functional requirements come directly from the five objectives in Section 1.8, and each is written as a testable assertion about behaviour that can be observed. Every one carries an FR number so the traceability matrix in Section 2.8 can map it to a design element and to the test case that verifies it. Requirements are stated as user-visible capabilities rather than as implementation mechanisms, so the streaming rebuild described in Chapter 7 changes how FR-2 is satisfied without changing the requirement.

- FR-1: The instructor shall be able to start a workshop session with a title, optional password, and optional file-share folder, and receive a shareable room code.
- FR-2: The system shall capture the instructor's screen and audio natively on Windows and Linux and stream it live to all connected students through an SFU relay.
- FR-3: The backend shall generate a live, continuously updated study guide from the instructor's spoken transcript and screen content.
- FR-4: Students shall be able to join a room using a room code, their name, and (if set) the session password, and receive a signed join token.
- FR-5: The system shall detect student code errors and explicit help requests and return AI-generated, context-grounded diagnoses/answers.
- FR-6: The instructor shall be able to share files (folder-watched, manually uploaded, or via URL) with students during a live session.
- FR-7: The system shall support in-session comprehension quizzes and proctored assessments with focus-loss tracking and automated scoring.
- FR-8: The system shall notify students when the instructor's session ends or the instructor temporarily disconnects, distinguishing the two cases.
- FR-9: Both applications shall support light and dark themes with persistent accent colours across views.

2.2 Non-Functional Requirements

- Performance: guide/quiz/error-diagnosis responses should return within a few seconds of the triggering event under normal Groq API latency.

- Reliability: student WebSocket connections should auto-reconnect on transient network drops without losing session state.
- Security: instructor privileges and student identity must be derived from a server-issued signed token, never from client-supplied parameters.
- Usability: joining a session as a student requires no installation: browser only.
- Scalability: the AI pipeline must not let a burst of simultaneous student requests overwhelm the upstream LLM provider's rate limit (handled via a per-session concurrency queue).
- Portability: the instructor app must build and run natively on Windows and Linux via Tauri v2, while students access the platform through standard evergreen browsers.

2.2.1 Prioritization of Non-Functional Requirements

Where non-functional requirements conflict, the order applied throughout the project was security first, then reliability, then usability, then performance, then scalability. Security comes first because a live session exposes the instructor's screen and voice and the identity of every participant. A performance regression can be recovered from; an access-control defect cannot. Reliability comes before usability because a session that drops is worse than a session that is awkward to configure. Scalability is last on purpose: the target for this phase is a single classroom batch, and optimising early for thousands of concurrent students would have distorted the architecture. The ordering shows up in the actual sequence of work, since the authentication rework was done before the streaming quality rebuild even though the rebuild is far more visible to users.

2.3 Assumptions and Constraints

- Instructor and students have a stable internet connection sufficient for live audio/video streaming.
- The instructor runs the desktop application on Windows or Linux; students need only a modern browser.
- AI features depend on an external LLM provider (Groq) being reachable and within its rate limits.
- Live delivery depends on a third-party relay. A room with no relay credentials, or one whose subscribe call fails, falls back to a direct peer mesh and inherits the mesh's scaling limit, so the fallback is a correctness guarantee rather than a capacity one.

2.4 Feasibility Study

2.4.1 Technical Feasibility

Each technical dependency was verified before adoption. Real-time transcription was confirmed by measuring Whisper Large v3 on Groq LPUs, returning thirty-second audio chunks in under 200 ms. Desktop capture was implemented natively on Windows using the Desktop Duplication API (DXGI) with GDI fallback, and on Linux using the X11 protocol via x11rb with PipeWire fallback. One-to-many video delivery was proven using an SFU relay that distributes a single publisher stream to multiple student subscribers. Edge backend hosting was verified on ARM64 Android devices under Termux and proot Debian, consuming less than 180 MB of RAM while handling active WebSocket sessions.

2.4.2 Economic Feasibility

The architecture operates at zero recurring cost during development and minimal expense in production. Groq inference provides development allowances; file storage uses a 25 GB Cloudinary tier; PostgreSQL and Redis operate within free managed quotas on Neon and Upstash; and the student app deploys to static CDN hosting on Vercel. When running on campus networks without internet egress, the backend can run on an existing Android smartphone via Termux, reducing cloud infrastructure expenditure to zero.

2.4.3 Operational Feasibility

Operational feasibility rests on minimal installation friction. Instructors install a single native binary, built and packaged automatically by GitHub Actions for Windows (NSIS executable, MSI installer) and Linux (.deb package, AppImage) with Google Drive download mirrors. Students install nothing, accessing sessions via a browser with a room code. Sessions require no on-premises server configuration or dedicated administration.

2.5 Software Requirements

Table 2.1 – Software Stack / Tooling

Layer	Technology	Reason
Instructor desktop	Tauri v2 (Rust + WebView2 on Windows, WebKitGTK on Linux)	8–15 MB install size, near-zero idle CPU, native screen capture on Windows (DXGI/GDI) and Linux (X11 x11rb / PipeWire)

Layer	Technology	Reason
Student web app	React 18 + TypeScript + Vite	Fast SPA, Monaco editor integration, zero-install for students
Backend	Node.js (ESM) + Express + ws	Lightweight; shares one WebSocket protocol with the Tauri instructor app
AI inference	Groq (LPU Inference Engine)	300+ tok/s; real-time generation practical at live workshop scale
LLM orchestration	LangChain + LangGraph	Retry loops, structured output parsing, session memory
Audio transcription	Whisper Large v3 via Groq	~180x realtime; a 30 s audio chunk transcribes in well under 200 ms
Text / diagnosis reasoning	Qwen3.8-27B (vision) and GPT-OSS-120B (text) via Groq	Sub-second vision parsing; strong reasoning and native tool-calling for Zod-validated outputs
Session memory	Redis (LangChain chat history)	Rolling 20-message context; survives brief backend restarts within TTL
Persistent storage	PostgreSQL (5 tables)	Guide blocks, files, error logs, session records
File storage	Cloudinary	Free 25 GB tier, direct student download URLs
State management	Zustand	Minimal boilerplate, no Provider-tree wrapping needed
Output validation	Zod	Runtime schema enforcement on LLM JSON output
Version control	Git / GitHub Actions	Automated multi-platform builds (Windows NSIS/MSI, Linux deb/AppImage), code signing, and Google Drive mirror fallback
Cloud / Hosting	Render (cloud), Vercel (frontend), Android Phone via Termux + Debian (edge)	Production cloud deployment or zero-cost local handset hosting

2.5.1 Prerequisites for Development

Node.js ≥ 20 , Rust + Cargo (via rustup), Docker Desktop (for local Postgres + Redis), a free Groq API key (console.groq.com, no card required), and a free-tier Cloudinary account (cloudinary.com, 25 GB).

2.6 Minimum System Configuration

Instructor Machine: Dual-core x86_64 or ARM64 processor, 4 GB RAM, 200 MB disk space, Windows 10/11 (64-bit) or Linux (Ubuntu 22.04+, Debian 12+, Fedora 38+ with X11 or Wayland), microphone, and 5 Mbps upstream network connection.

Student Machine: Device running a modern browser (Chrome 110+, Firefox 115+, Safari 16+, Edge 110+), 2 GB RAM, and 2 Mbps downstream connection. Edge Host (Optional): Android phone running Android 7.0+, 64-bit ARM CPU, 2 GB free storage, and Termux with proot Debian.

2.7 External Interface Requirements

2.7.1 User Interfaces

The instructor application has four surfaces: a session-setup form, a live session panel with the student roster and file-sharing controls, a settings page grouping appearance, profile, inference key, capture and recording configuration, and a system-tray presence that lets the application run without taking screen space while teaching. The student application has a landing page explaining how joining works, a join form, and a live session view that combines the video panel, the generated guide, the exercise editor and terminal, and the assistance panel. Both applications implement the same theming contract, so a user moving between them sees consistent typography, spacing and accent behaviour.

2.7.2 Software Interfaces

Four external software interfaces are required. The inference provider is reached over HTTPS for both speech transcription and text generation, authenticated with an API key held in server configuration and never sent to a client [11]. The media platform is reached for ingress creation and subscriber-token issuance, authenticated with a signed credential pair [10]. The file-storage service is reached for upload and returns a public download URL that is broadcast to students. The relational database and the in-memory store are reached over their native protocols from the backend only. Neither is reachable from a client, and no client ever holds a database credential.

2.7.3 Communication Interfaces

Control-plane communication uses a single WebSocket endpoint carrying JSON messages, with the connection authenticated by a signed token supplied as a query parameter and

validated before the upgrade completes. Request-response operations use JSON over HTTPS on the REST routes listed in Section 3.3. Media travels on a separate path: currently peer connections negotiated through the WebSocket relay [14], and under the planned rebuild an ingest protocol to the media platform with per-student subscribe tokens [10]. Keeping media off the control channel is a requirement rather than a happy accident, because it is what allows the streaming implementation to be replaced without renegotiating the application protocol.

2.8 Requirements Traceability Matrix

The matrix below maps each functional requirement to the design section that specifies it and to the test case in Section 6.2 that verifies it. Requirements marked partially verified are those whose implementation is complete but whose quality is bounded by something outside the code: the inference allowance on a free tier, or the availability of the relay.

Table 2.2 – Requirements Traceability Matrix

FR	Requirement (abridged)	Design Section	Verified By	Status
FR-1	Instructor starts session, receives room code	5.1, 7.2	TC-5, Figure 5.7	Verified
FR-2	Live screen + audio streamed natively (Windows & Linux) via SFU	5.1, 5.4, 7.2	Manual E2E	Partially verified
FR-3	Live guide generated from transcript and screen	5.3, 5.5, 7.2	Manual E2E	Verified
FR-4	Student joins by room code and receives signed token	5.4, 7.2	TC-1, TC-5, Figure 5.2	Verified
FR-5	Error detection and grounded AI diagnosis	5.5, 7.2	TC-4	Verified
FR-6	Instructor shares files during a live session	5.4, 7.2	TC-6	Verified
FR-7	In-session comprehension quizzes & proctored assessments	5.5, 7.2	Manual E2E	Verified
FR-8	Session-end and instructor-offline notification	5.4, 7.2	TC-2, TC-3	Verified
FR-9	Light/dark theming with persisted accent	7.2	TC-7, Figure 5.8	Verified

2.9 Expected Outcomes

The phase was planned against five checkable outcomes: a live session an instructor can start and students can join from a browser with no installation; a study guide generated from the session itself and visible to students while the session is still running; an error-diagnosis path that takes a student's failure text and returns a grounded explanation and a corrective action; an

authentication model in which no client-supplied parameter can influence role or identity; and a consistent, accessible interface across both applications. Chapter 6 evaluates the delivered system against each of these, and Chapter 7 states plainly which are complete and which remain bounded by the streaming pipeline still being rebuilt.

Chapter 3

HIGH-LEVEL DESIGN

3.1 System Architecture

WKAI is composed of three independent applications sharing one backend contract:

- Instructor app (wkai/): Tauri v2 (Rust + React, TypeScript, Vite). The Rust core manages OS-level screen capture (DXGI and GDI on Windows; native X11 via x11rb and PipeWire on Linux), audio capture, tray lifecycle, and local recording.
- Student app (wkai-student/): React with TypeScript and Vite, a zero-install browser application holding a Monaco editor and an xterm terminal for sandboxed exercises, the live view, the generated guide, the AI help panel, the private question thread and a proctored assessment runner. The same build also serves the public landing and download pages.
- Backend (wkai-backend/): Node.js with ES modules, Express and ws, backed by PostgreSQL for durable records and Redis for room state and session memory. The backend deploys to cloud containers (Render) or local edge hardware (Android Termux + Debian proot).

Instructor capture runs entirely outside the browser runtime. The Rust capture module grabs framebuffers per operating system (DXGI on Windows, x11rb on Linux), executes SIMD colour conversion, and feeds the WebRTC SFU streaming pipeline.

Seen as layers rather than as applications, the system has five. The capture layer is native code inside the instructor application, responsible only for acquiring frames and audio and passing them on over a bounded channel. The transport layer is the WebSocket control channel plus the separate media path, and it is the only layer both client applications share. The orchestration layer is the backend's agent runtime, which owns prompt construction, state-machine execution, retry policy and concurrency control. The persistence layer separates durable records in the relational store from ephemeral, time-bounded state in the in-memory store. The presentation layer is the two user interfaces, which hold no authoritative state of their own and render only from messages received on the control channel and snapshots fetched over REST. Two invariants hold across those layers, and they account for most of the design's resilience. First, no client is authoritative about anything. A client's role, its session membership and its identity are all properties of the server-issued token it presents, and a client that reconnects receives a full state snapshot instead of replaying local assumptions. Second, the control path and the media path are independent. A student whose media connection degrades keeps

receiving guide blocks, quizzes and diagnoses over the control channel, and the media pipeline was in fact replaced wholesale during this phase — browser capture and a peer mesh gave way to native capture and a relay — without one change to the application protocol. Neither invariant was present in the original design. Both came out of the hardening work described in Chapter 5, and the second is what made the streaming rebuild a contained piece of work rather than a rewrite.

The deployment view differs from the logical view in one respect worth stating. The backend is described as a single service, but it does three quite different kinds of work: low-latency message relay, database-bound request handling, and long-running outbound inference calls. In the current deployment all three share one process. That is acceptable at classroom scale, but it means a slow inference call competes for event-loop attention with message relay, which is part of why the per-session concurrency queue exists. Splitting the inference workers out of the relay process is named in Chapter 7 as a scaling step, and the agent contract was written with that split in mind: agents are invoked through a uniform interface and hold no references to connection objects, so moving them into a separate process is a deployment change rather than a rewrite.

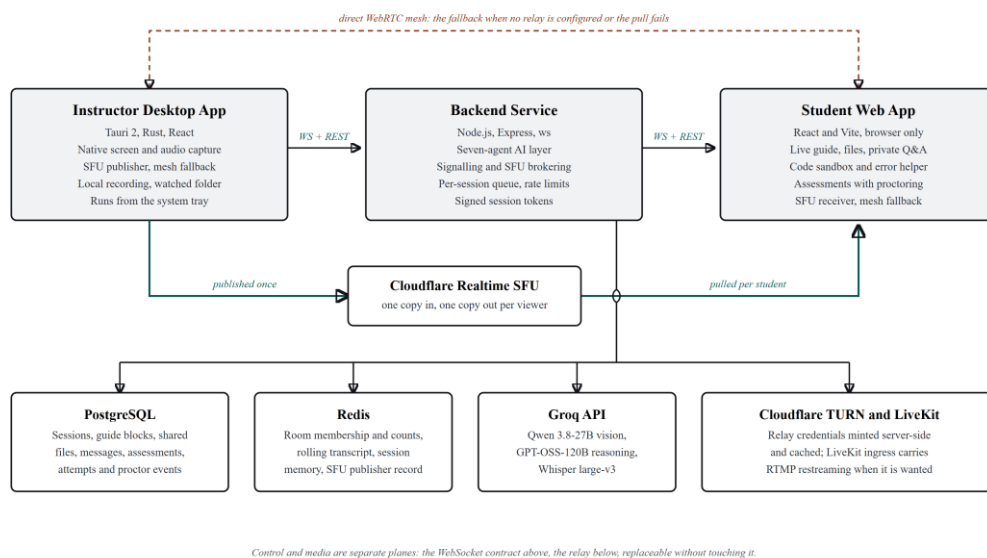


Figure 3.1 – WKAI system architecture: the three applications and their backing services

3.2 Software Design

Use Case Diagram (textual): Two primary actors: Instructor and Student. The Instructor actor: creates/starts a session, streams screen+audio, shares files, ends the session, configures AI/capture/recording settings and appearance. The Student actor: joins a session with a room

code (and password if set), views the live stream and auto-generated guide, runs code in the sandboxed editor/terminal, requests AI help on errors, answers comprehension quizzes, receives shared files, and messages the instructor.

Data flow at a high level: the instructor's microphone and screen feed transcript and frame ingestion in the backend, which runs the LangGraph agents [12] through classify, then diagnose, explain or generate. Two gates sit in that path and reject rather than pass: a transcript gate that discards stock Whisper hallucinations, pure filler and decoder loops before a model is billed for them, and a grounding filter that drops guide blocks which only narrate the screen, carry too little to teach, or claim code while carrying none. Session memory in Redis keeps a rolling window of what has already been taught, extended across a course by an optional workspace, and the surviving guide, quiz and diagnosis messages are broadcast over the WebSocket to the student application, which renders them live.

Database design: PostgreSQL holds the persistent tables covering sessions and workspaces, guide blocks, comprehension questions, shared files, student messages, error resolutions, and the assessment tables — the assessment itself, its questions, each attempt, each answer, and the proctoring events recorded during an attempt. Redis holds the fast-changing state: active room membership and student counts, the rolling transcript, conversational memory for the agents, the session snapshot consulted during token validation, and the record of where the instructor's stream currently lives on the relay.

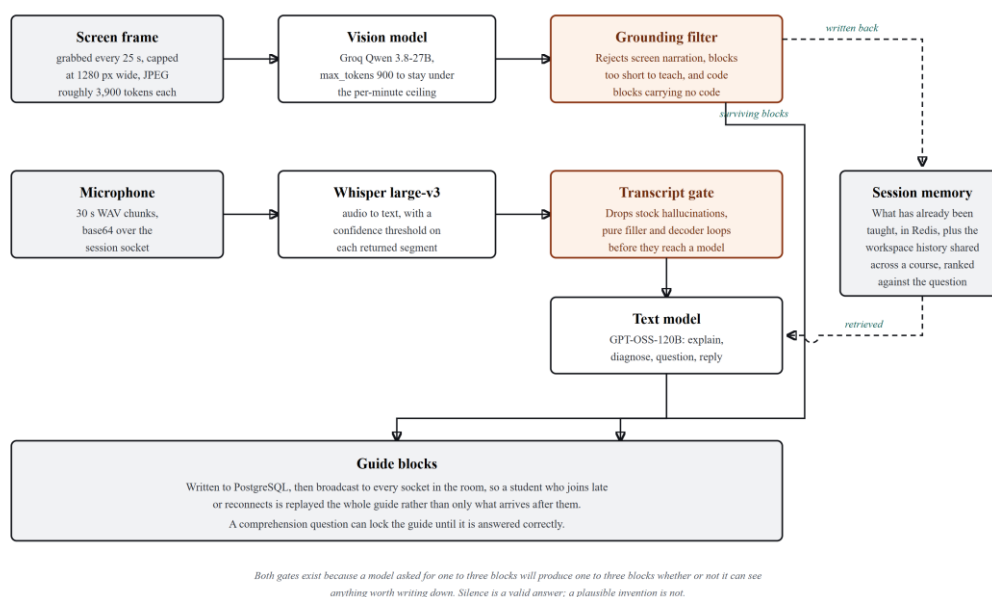


Figure 3.2 – Capture-to-guide AI pipeline

3.2.1 Persistent Data Model

The relational schema is deliberately small. It records what has to survive the session, meaning who ran it, what was taught, what was shared, what students asked and what went wrong, and nothing that can be reconstructed. There is no table of connected clients, because connection state is inherently ephemeral and belongs in the in-memory store. An earlier design that persisted client rows left stale records after every restart and was removed.

Table 3.1 – Persistent (PostgreSQL) Data Model

Table	Purpose	Key Fields	Lifetime
sessions	One row per workshop session, optionally tied to a workspace	id, room_code, instructor_name, workshop_title, status, workspace_id, started_at, ended_at	Permanent
workspaces	A named course or cohort whose sessions share AI memory	id, name, created_at	Permanent
guide_blocks	Guide cards generated during a session, in order	id, session_id, type, title, content, code, language, locked, created_at	Permanent
comprehension_questions	Checks generated from recent guide content	id, session_id, guide_block_id, question, options, correct_index, explanation	Permanent
shared_files	Files shared by the instructor, with storage URLs	id, session_id, name, url, size_bytes, shared_at	Permanent
messages	Student questions and the instructor or AI reply	id, session_id, message_id, student_id, student_name, message, reply, reply_role, replied_at	Permanent
error_resolutions	Errors submitted for diagnosis and what was returned	id, session_id, student_id, error_text, diagnosis, fix_command, created_at	Permanent
assessments	A quiz or timed test authored for a session	id, session_id, title, mode, duration_seconds, status	Permanent

Table	Purpose	Key Fields	Lifetime
assessment_questions	The questions belonging to one assessment	id, assessment_id, prompt, options, correct_index, marks	Permanent
assessment_attempts	One student's run at an assessment	id, assessment_id, student_id, student_name, started_at, submitted_at, score	Permanent
assessment_answers	The answer given to each question in an attempt	id, attempt_id, question_id, selected_index, is_correct	Permanent
assessment_events	Proctoring signals: focus loss, paste, visibility change	id, attempt_id, kind, detail, created_at	Permanent

3.2.2 Ephemeral State Model

The in-memory store holds three kinds of state, each with an explicit expiry. Active-room state maps a room code to its connected participants and current status, and the relay consults it on every broadcast. Conversational memory keeps a rolling window of the twenty most recent messages for a session, which is what lets an agent invoked late in a session know what was already taught without re-reading the whole transcript; the window size was chosen empirically, at the point where more context stopped improving diagnosis quality and kept adding prompt cost and latency. Token-validation support holds revocation and session-liveness information used during connection authentication. All three carry a seven-day time to live, long enough for a course that runs across several days, so an abandoned session still clears itself with no cleanup task.

Module Structure of the Backend: Routes are partitioned into session management, authentication, file sharing, assessments, and AI orchestration. The ai/ package defines the LangGraph state machines, while wkai-mcp exposes tool definitions for external autonomous agents. The assessments/ module handles quiz dispatch, submission evaluation, and student focus-loss event aggregation.

The backend source is organised by responsibility rather than by technical type. The routes directory holds thin HTTP handlers that validate input, delegate and serialise a response, with no business logic and no direct model calls. The ai directory holds two sub-layers: agents, one file per agent implementing the shared lifecycle contract, and graphs, the state machines those agents execute. The websocket directory holds the connection authenticator, the room registry

and the broadcast helpers. The db directory holds the schema definition and the query functions, and no query text appears outside it. The livekit directory holds the dependency-free media-platform client, including token signing and the calls needed to create and tear down a session ingress. The layout keeps the two most volatile areas, prompt logic and media integration, each behind a single directory boundary.

3.2.4 Client Application Structure

Both client applications use the same three-tier internal structure: a store tier holding application state, a service tier owning all input and output, and a component tier that renders and dispatches. State lives in a small store rather than in component-local state, so a message arriving on the control channel can update every dependent view without threading props, and so theme and session state can be persisted and rehydrated on load. All network access sits in the service tier, which means a protocol change touches one file per client instead of every component that needs data. The instructor application adds a fourth concern the student application does not have, an inter-process bridge to its native layer, kept behind a guard so the interface can be developed and previewed in an ordinary browser without a native build. That guard is a development convenience rather than a shipping path: the screenshots in Section 5.5 were taken from the packaged application in a live session, because a preview in a browser cannot show native capture doing its job.

3.3 Interface between Components (Protocol)

All real-time communication uses one WebSocket endpoint of the form `ws://host/ws?token=...`. The token is server-issued and signed, valid for twelve hours for an instructor and eight for a student join. Role, session ID, student ID and student name are all derived from it on the server, and no other query parameter is trusted, which is what closes the role and identity spoofing hole described in Section 4.2. A connection without a valid token is rejected outright.

Table 3.2 – Key WebSocket Message Types

Message Type	Direction	Description
audio-transcript	instructor → server	Whisper transcript text → Redis memory + intent agent
guide-block	server → students	AI-generated guide card
comprehension-question / -answer / -result	server ↔ students	Comprehension-gate quiz flow
file-shared	server → students	Cloudinary URL broadcast

Message Type	Direction	Description
student-error	student → server	Error text → LangGraph error agent (queued per session)
error-resolved	server → student	Diagnosis + fix command, with graceful fallback on failure
student-joined / student-left	server → room	Updated student count
student-list	server → instructor	Full student roster on join
session-ended	server → room	Instructor ended session
session-state	server → new client	Full state snapshot on connect
share-intent-detected	server → instructor	LangGraph detected a “share this file” spoken intent
webrtc-offer / -answer / -ice-candidate / -request-offer / -session-reset	instructor ↔ student	WebRTC signalling relay, per-student targeted
assessment-published / -progress / -submitted	server ↔ room	Assessment lifecycle: released to students, progress back to the instructor, final submission
assessment-violation	student → server	Proctoring signal recorded against the attempt
student-removed	server → room	A student was removed by the instructor, and is told why
instructor-online / -offline	server → students	Presence, with an eight-second grace period

Table 3.3 – Key Backend REST Routes

Method	Path	Description
POST	/api/sessions	Create session; returns rate-limited instructorToken
POST	/api/sessions/:roomCode/join	Password check; server assigns studentId, returns joinToken
GET	/api/sessions/:roomCode	Validate a room code (public)
PATCH	/api/sessions/:id/end	End session + cleanup — instructor-token only
GET	/api/sessions/:id/guide	Fetch all guide blocks — any valid session token
POST	/api/ai/transcribe	Groq Whisper audio → text
POST	/api/ai/diagnose	LangGraph error-diagnosis agent
POST	/api/ai/intent	LangGraph intent-detection agent

Method	Path	Description
POST	/api/ai/colab-assist	Colab-assist agent (analyzes pasted notebook/output)
GET	/api/ai/agents	Agent health + metrics snapshot
POST	/api/files/upload	Upload to Cloudinary
POST	/api/run	Sandboxed code execution (python3 / node / bash)
GET	/api/webrtc/:sessionId/ice	ICE servers, including TURN credentials minted per request
GET	/api/webrtc/:sessionId/sfu/status	Whether the room currently has a publisher on the relay
POST	/api/webrtc/:sessionId/sfu/publish	Instructor negotiates a publish session against the SFU
POST	/api/webrtc/:sessionId/sfu/subscribe	Student pulls the published track — session-token gated
POST	/api/assessments	Create an assessment with its questions
POST	/api/assessments/:id/attempts	Start an attempt; returns the questions without the answers
POST	/api/assessments/:id/submit	Submit an attempt; scoring happens on the server
GET	/api/workspaces	List workspaces, for the setup screen
GET	/api/network-info	The LAN address students on the same network can actually reach

Live media travels on an independent WebRTC transport. The instructor negotiates a publish session against the SFU relay using backend-brokered credentials. Students connect as subscribers, pulling video tracks over UDP/SRTP without loading the instructor's local uplink. If the relay is unavailable, connections drop back to a direct peer mesh.

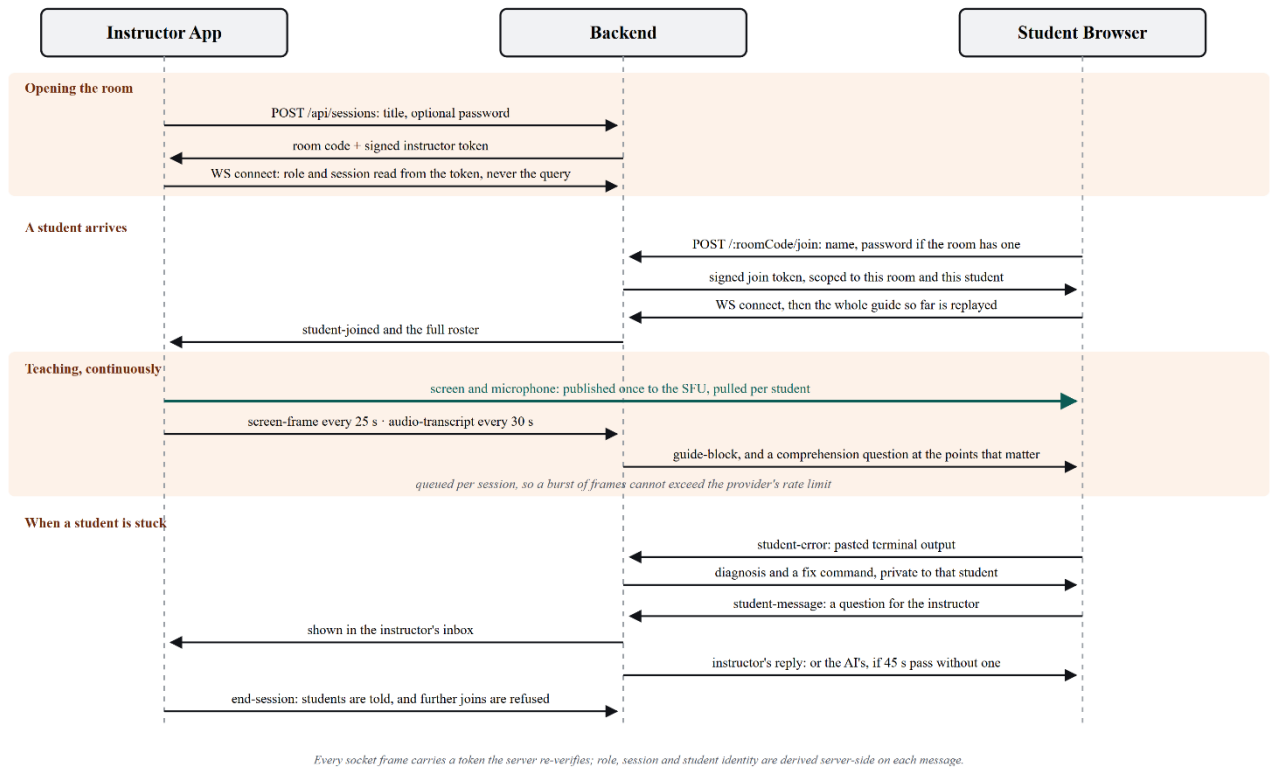


Figure 3.3 – Session message sequence across the three applications

3.4 Design Patterns Applied

Several established patterns were adopted deliberately, and naming them makes the intent legible. The agent layer is a strategy pattern behind a uniform contract: callers pick a capability by name and never branch on which model or graph implements it. The state machine inside each agent is an explicit pipeline, where each node performs one transformation on a shared state object and no node calls another directly, which is what makes retry and partial failure tractable. The real-time layer is publish and subscribe, with the room registry as broker and clients as subscribers identified by verified token claims. The client stores are observers, so one inbound message updates every dependent view without explicit propagation. The theming engine is a computed derivation: one user-chosen accent value generates its own contrast-safe foreground and text derivatives at runtime, so there is one value to keep consistent rather than three per theme.

One pattern was rejected. An event-sourced session model, persisting every message and rebuilding state by replay, was considered because it would have solved restart resilience elegantly. It was rejected for this phase on scope: it needed a rewrite of the persistence layer to buy a reliability property that moving room state into the in-memory store also delivers. It is

recorded here because it remains the better long-term answer if the system is scaled across several backend instances.

3.5 Design Trade-offs and Rationale

Four trade-offs shaped the delivered design. The first was native capture against browser capture. Browser capture was chosen first because it is portable and needs no per-operating-system code, and it is why a working end-to-end system existed by the end of the first phase. Its cost was that capture quality was bounded by browser APIs and that one encode ran per connected student. That trade has since been reversed with evidence: the portable path established the protocol, and the native path optimised it without changing it.

The second was a per-student connection mesh against a server-side relay. The mesh requires no media infrastructure and works immediately, but its upstream cost grows linearly with cohort size, which makes the instructor's network the scaling limit: thirty students means thirty encoders and thirty copies of the same screen leaving one wireless uplink. The relay moves that cost to a service the project does not operate but must integrate and authenticate against. Both exist in the delivered system, the relay as the normal path and the mesh as the fallback, which keeps a room usable when the relay is unreachable or unconfigured.

The third was durable against ephemeral state. Keeping room membership in process memory made the relay simple and fast, and made a restart destructive. Keeping it in the in-memory store makes a restart survivable, at the cost of a network round trip on every broadcast. The current design took speed and simplicity, recorded the consequence as a limitation, and scheduled the reversal.

The fourth was model capability against latency and cost. A larger model produces better diagnoses; a smaller one responds faster and permits more invocations per session. The resolution was to differentiate by task rather than choose globally: cheap lexical pre-filters and classification steps run ahead of expensive generation, so the largest model is invoked only when its output will be shown to a student. Hence the intent agent's heuristic first node, and the error agent classifying before it diagnoses.

Chapter 4

DETAILED DESIGN

4.1 AI Agent Layer Design

The AI agent layer is hosted entirely on Groq's LPU inference infrastructure to sustain the low latencies required for live technical instruction. Speech transcription uses Whisper Large v3, transcribing thirty-second audio chunks in under 200 ms. Frame vision processing uses Qwen3.8-27B (qwen/qwen3.8-27b, max_tokens=900), which replaced the decommissioned llama-4-scout and operates under Groq's output-tokens-per-minute ceiling without emitting disruptive reasoning blocks. Text reasoning, error diagnosis, intent detection, and assessment authoring use GPT-OSS-120B (openai/gpt-oss-120b), which migrated from llama-3.3-70b-variant for superior structured output generation and reliable Zod schema validation. A dedicated creative instance of GPT-OSS-120B (temperature 0.6, max_tokens=1200) generates comprehension check questions.

4.1.1 Common Agent Contract

Each agent exposes four members. A name identifies it in logs, metrics and the health endpoint. A version is incremented whenever the prompt or the graph structure changes, so a behavioural regression can be attributed to a revision instead of guessed at. An invoke method takes a typed input object and returns a validated output object, and it is the only entry point; it builds its own prompt from the session context it is given, so no caller needs to know what the model is asked. A health check reports reachability and recent error rate without running a billable generation, which is what makes the aggregate status endpoint cheap to poll. Every agent is also gated by an environment flag, so a capability can be switched off in a deployment without removing its code. That flag was used during development to find out whether an observed latency problem came from one particular agent.

4.1.2 Error Diagnosis Agent

The error agent is the most heavily used and the most carefully structured. It runs a three-node state machine: classify, diagnose, parse. Classification works out the category of failure from the raw error text, whether that is a missing dependency, a syntax error, an environment or path problem, a permission failure, a runtime exception or something unrecognised, because the category decides both how the diagnosis is framed and whether a corrective command can be proposed at all. Diagnosis then generates an explanation and, where one applies, a single

corrective command, with the prompt built from the error text, the classification and the rolling session context, so the explanation refers to what the class was actually doing rather than to a generic case. Parsing validates the structured output against a schema. On a validation failure the graph retries a bounded number of times and then returns a degraded but honest response, which is better than emitting unvalidated text that the student interface would render as authoritative [9].

4.1.3 Intent Detection Agent

The intent agent listens to the transcript and picks out statements implying an action the instructor expects the system to take, most often an intention to share a file. It runs a heuristic pre-filter, then a classification node, then a matching node. The pre-filter exists purely for cost control: almost every transcript segment contains no actionable intent, and sending all of them to a model would multiply inference volume tenfold for no gain. Only segments that pass a cheap lexical test go on to classification. When an intent is confirmed, the matching node tries to resolve the referenced object against the instructor's watched folder, and the result reaches the instructor as a prompt to confirm rather than as an automatic action. That is deliberate. A false positive that shares the wrong file during a live class is a serious failure, while a false positive that shows a dismissible suggestion is only noise.

4.1.4 Transcript Explanation Agent

This agent turns transcript segments into guide blocks. Its central constraint comes from the lecture-understanding literature reviewed in Section 1.6: it has to summarise at segment granularity and publish as it goes, rather than maintain one continuously rewritten document [7]. Publishing per segment buys three things. A student joining mid-session gets a coherent history instead of a partial rewrite, a badly generated block can be identified and superseded on its own, and the guide renders progressively so content appears while the session is still running. Each block carries a sequence number, a heading, a body, and a provenance marker recording whether it came from speech, from screen content, or from both.

4.1.5 Comprehension Coaching Agent

The coaching agent generates comprehension questions from recently published guide content and evaluates the answers. Its design follows the elicit-before-explain constraint taken from the conversational-tutoring literature [2]: the agent may not include the answer in the question payload sent to the client, because that would put the answer in the browser where anyone can

read it, so evaluation happens on the server. Progression gating is a property of how the client renders guide state rather than of data withheld from it, and that is acknowledged here as a soft control, since a determined student can inspect the client. Real enforcement needs the native companion application described in Chapter 7, and the limitation is stated rather than overclaimed.

4.1.6 Student Messaging Agent

The messaging agent handles direct student questions that are not tied to an execution failure. It differs from the error agent in that its input is a natural-language question rather than machine-generated failure text, so classification is about whether the question can be answered from session context, answered from general knowledge, or properly belongs to the instructor. Questions in the third category go to the instructor's inbox instead of being answered, which keeps the instructor's authority over course-specific matters such as assessment and scope. That routing behaviour, not answer quality, was what most of the iteration on this agent was about.

4.1.7 Notebook Assistance Agent

The notebook assistance agent addresses a workflow that came up repeatedly: students work in hosted notebook environments whose output cells hold both the failure and a large amount of irrelevant surrounding output. The agent takes notebook content, isolates the relevant cell and its output, and produces a diagnosis in the same shape as the error agent's, so the client renders both through one component. It is kept separate from the error agent because its input preprocessing is substantial and notebook-specific. Merging the two would have made the error agent's prompt conditional on where its input came from, which is exactly the branching the agent contract exists to prevent.

4.1.8 Resource Access Agent

The resource access agent resolves references to external material, such as a documentation page or a link mentioned during the session, and returns a summarised, session-relevant extract rather than the raw page. It exists to prevent a common failure where a student is sent to a long reference document mid-exercise and loses the thread of the session. It is the least used of the agents and is feature-flagged off by default, because its value depends on how the instructor teaches.

4.1.9 Concurrency and Rate Control

All agent invocations for a session pass through a queue with bounded concurrency. It exists because student-triggered invocations are bursty by nature: a class following the same instruction hits the same failure within seconds of each other, producing a spike of simultaneous diagnosis requests. Without the queue that spike goes straight to the upstream provider and is rejected as a rate-limit violation, so every student gets an error instead of every student getting a slightly delayed answer [11]. The queue turns a hard failure into bounded latency, which is the right trade for this workload. Queue depth and wait time are published on the agent metrics endpoint, and watching them under load is what established that per-session scope was enough, without a global queue.

4.1.10 Structured Output Validation

Every agent output is validated at runtime against an explicit schema before it leaves the AI layer. This is a security and correctness boundary, not a convenience. Model output is untrusted input, and a client that renders unvalidated model text is exposed both to malformed structures that break rendering and to content that was never meant to reach a student. Validation failures are counted per agent and per version, which turns prompt regressions into a metric instead of scattered user reports.

Table 4.1 – AI Agent Layer: Triggers, Contracts, and Failure Behaviour

Agent	Trigger	Input	Output	Failure Behaviour
Error Diagnosis	Student execution failure	Error text + session context	Category, explanation, fix command	Bounded retry, then degraded response
Intent Detection	Transcript segment passes pre-filter	Transcript segment + folder listing	Intent type + matched file	Silently ignored; no prompt shown
Transcript Explanation	New transcript segment	Segment + recent guide state	Guide block (heading, body, provenance)	Segment skipped; sequence preserved
Comprehension Coaching	Guide block published	Recent guide content	Question; server-side answer evaluation	Quiz suppressed for that block
Student Messaging	Student sends a question	Question + session context	Answer, or escalation to instructor	Escalated to instructor inbox
Notebook Assistance	Student pastes notebook output	Notebook cell + output	Diagnosis in error-agent shape	Falls back to error agent
Resource Access	Reference mentioned or requested	URL or reference string	Session-relevant summarized extract	Disabled by default; returns nothing

4.2 Security Architecture

The security boundary derives all privileges from cryptographically signed tokens. In addition to role enforcement, the backend supports explicit student removal with mandatory reason logging, broadcasting eviction notices over WebSockets. Forgotten room passwords can be reset by the instructor using a server-issued cryptographic access key.

4.2.1 Token-Based Identity

The replacement model issues two token types, both signed on the server and both carrying the claims the server will later rely on. An instructor token is issued at session creation to whoever created the session and lasts twelve hours, a window chosen to cover a teaching day without re-authentication mid-session. A student join token is issued only after the server has checked the room code and, where one is set, the session password, and it lasts eight hours. Every real-time connection has to present a token, and role, session identifier, student identifier and display name are read from the verified token and nothing else. Client-supplied role and session parameters are never read rather than merely distrusted, and that distinction matters: a validation check can be bypassed by a case its author did not anticipate, but a parameter that is never read cannot.

4.2.2 Authorization Boundaries

Privileged operations are gated on token type rather than on session membership. Ending a session, requesting a media ingress and sharing files all require an instructor token for that specific session, and holding a valid student token for the same session is not enough. Read operations that expose only session-scoped content, such as fetching the accumulated guide, accept either token type. Public operations are limited to one route that reports whether a room code exists, which the join form needs and which reveals nothing else: not the title, not the participant count, not whether a password is set.

4.2.3 Supporting Controls

Three further controls were added alongside the token rework. Rate limiting was applied to the session creation and join routes, since both are necessarily unauthenticated and are the natural target for enumerating room codes. Token comparison was made timing-safe, since a naive comparison leaks through its execution time how much of a candidate value is correct. The backend also refuses to start in production when its signing secret is unset, turning a deployment that signs tokens with an empty or default key from a silent vulnerability into a

startup failure. Secrets, including the inference provider key, live only in server configuration and are never sent to any client.

4.2.4 Residual Risks

Two limitations remain. Comprehension gating is enforced in the client and is therefore advisory; a student willing to inspect the application can bypass it, and real enforcement needs the native companion application described in Chapter 7. Room and session state also lives in process memory, so a backend restart invalidates active room state even though the tokens stay valid, and students have to rejoin. Moving that state into the in-memory store is planned work, recorded in the risk register in Appendix F.

Chapter 5

IMPLEMENTATION

5.1 Development Environment Setup

Backend: Node.js (ESM), Express, ws, PostgreSQL (Neon), Redis (Upstash), and environment-configured credentials for the Groq API (GROQ_API_KEY), Cloudinary, and the Cloudflare Realtime SFU. Build dependencies for the desktop app include Rust 1.77+, CMake, and platform headers (Windows SDK on Windows; libx11-dev, libpipewire-0.3-dev, and mesa-common-dev on Linux).

5.2 Module-wise Implementation

AI agent layer (wkai-backend/src/ai/): a central Agents/ layer defines the common contract of name, version, invoke and health check, implemented by VoiceAgent, QuizAgent, DebugAgent, IntentAgent, MessageAgent, ColabAgent and UrlAccessAgent. Each is feature-flagged through an environment variable and reported at GET /api/ai/agents for health and metrics. Underneath, graphs/ holds the LangGraph state machines [12]: errorAgent.js runs classify, diagnose and parse with capped retries; intentAgent.js runs a heuristic, then classify-intent, then match-file, which is how the system notices the instructor saying something like “I’ll share this file”; and there are further graphs for transcript explanation, comprehension coaching, messaging, notebook assistance and assessment authoring. Two pure modules sit beside them and hold the rules that decide what is allowed through: transcriptQuality.js for speech and guideBlockFilter.js for generated blocks. They are deliberately free of any model import, which is what lets them be unit-tested without an API key.

Security hardening was completed during this phase. The original design took a connecting WebSocket client's role from a client-supplied query parameter (?role=instructor), which meant any client could claim instructor privileges over a session it did not own. The fix was a full signed-token rework. sessionAccess.js now issues two token types, issueInstructorToken valid for twelve hours and issueStudentJoinToken valid for eight, and every WebSocket connection must present a valid ?token= from which role, session ID and student identity are derived on the server. Client-supplied role and session parameters are no longer trusted. Alongside that, rate limiting was added to the join and create routes, a per-session AI concurrency queue was added so simultaneous student requests cannot exceed the provider rate

limit [11], timing-safe comparison replaced a naive string comparison in token verification, and the backend now refuses to boot in production if its signing secret is unset.

Instructor capture pipeline: the Rust `native_capture` module implements screen capture behind a common backend trait. On Windows, it leverages the Desktop Duplication API (DXGI) for hardware-accelerated frame grabbing, falling back to GDI if hardware acceleration is unavailable. On Linux, it implements direct X11 capture via `x11rb`, with a `getDisplayMedia` webview fallback for Wayland desktops. Frames are converted to BGRA/RGBA using SIMD routines before delivery to the WebRTC video track.

Streaming backend: media delivery is brokered by the backend so that no client holds provider credentials. The backend provides short-lived ICE servers and room tokens, allowing the instructor to publish one video track and students to subscribe via the SFU relay. If relay credentials are absent, clients fall back to a direct WebRTC mesh.

Session lifecycle correctness: two ordering and robustness bugs found late in the phase were fixed. The backend was broadcasting session-ended after tearing down the WebSocket room, so the broadcast reached zero clients and students were never told the session had ended. Separately, a connection attempt to an already-ended room returned a generic error that clients treated as transient, which produced an unbounded three-second reconnect loop and a sustained database query storm. An instructor online and offline signal with an eight-second grace period was added, so a brief reconnect such as a page reload or a network blip does not alarm students while a genuine departure still does.

Theming, in both applications: a CSS-variable theme engine switches an accent, accent-text and accent-foreground trio computed by luminance, so any accent colour a user picks stays legible in light and dark mode. `initTheme()` runs before paint to avoid a flash of the wrong theme, and the state is a persisted Zustand store. The base palette was redesigned from indigo on slate to teal on zinc. The instructor application also gained an `isTauri` guard so its React interface can be developed and previewed in a plain browser without a full Tauri and Cargo build.

5.2.1 Student Exercise Sandbox

Student Exercise Sandbox and Assessments: The student application embeds Monaco editor and xterm instances for running exercises in Python, JavaScript, and TypeScript. In addition, an assessment runner presents timed comprehension quizzes and coding challenges. The client tracks focus-loss ('away modal') and paste events, sending telemetry to the backend to support academic integrity auditing.

5.2.2 File Sharing Pipeline

File Sharing and Colab Context Ingestion: File sharing supports drag-and-drop uploads, folder-watching on the instructor desktop, and automated Cloudinary hosting. In addition, the backend can ingest Google Colab notebooks, extracting code cells, execution outputs, and markdown notes into the session workspace context to guide AI explanations.

5.2.3 Guide Delivery and Reconnection

Guide blocks are delivered as individual messages as they are generated, and are also retrievable in bulk over a REST route. The duality makes reconnection simple: a client that reconnects does not try to reconcile a partial stream, it asks for a complete state snapshot and re-renders from it, discarding whatever it held. The same snapshot serves students who join after a session has started, so late joiners and reconnecting clients run identical code paths. That was a deliberate simplification, because handling the two cases differently was the source of an early class of state-mismatch defects.

5.2.4 Instructor Presence Signalling

Session Controls and Presence Signalling: The instructor interface features a floating PresentBar that stays accessible during screen sharing, complete with tray minimization, participant count indicators, student kick controls with audit logs, and password recovery tools.

5.3 Build, Release, and Deployment Automation

Three deployment targets are automated using GitHub Actions. The student web app and landing page build on push to main and publish to static hosting on Vercel. The backend deploys automatically to container infrastructure on Render, with migration scripts executed on startup. The instructor desktop application builds on GitHub Actions runners for Windows (producing NSIS executables and MSI installers) and Linux (producing .deb packages and AppImages on ubuntu-24.04). Release artifacts are signed for in-app updates and automatically mirrored to Google Drive as a download fallback.

One development-workflow investment deserves a mention because it changed the pace of interface work. The instructor application is a native shell hosting a web interface, and by default that interface cannot run without a full native build, a slow cycle that made iterative styling expensive. A guard was added that detects whether the native bridge is present and substitutes inert behaviour when it is not, which lets the whole interface be developed and

previewed in an ordinary browser. It cut the feedback loop for interface changes from minutes to seconds, and it is why a styling change no longer costs a Cargo build.

5.4 Implementation Challenges Encountered

Four classes of difficulty required focused engineering this phase. The first was Linux display server diversity: capturing screen frames reliably across X11 and Wayland required implementing direct X11 protocol integration via x11rb while providing a PipeWire and webview capture fallback for Wayland. The second was process lifecycle management on mobile edge devices, where Android 12+ phantom process monitors aggressively terminate background proot child processes, requiring explicit wake-locks and child process monitor adjustments.

The second class is negotiation races. Establishing a peer connection while the source stream has not yet produced its first frame yields a connection that completes successfully and carries nothing. This was resolved by gating negotiation on the arrival of a real frame rather than on the existence of a stream object, which is a stricter and more meaningful precondition.

The third class is lifecycle ordering, discussed in Section 5.2.4: broadcasting after teardown, and treating a terminal state as retryable. The second of those was the most damaging defect of the phase in operational terms, because a client that reads a permanent condition as transient will retry forever, and a population of such clients produces a sustained load spike against the database that looks like an infrastructure problem rather than a client-logic problem. The durable fix is to distinguish terminal from transient conditions in the protocol response instead of leaving that judgement to the client.

The fourth class is upstream change. A hosted model in use during the phase was deprecated by its provider and had to be replaced [11]. Because model selection was already isolated to configuration inside the agent layer rather than embedded in call sites, the migration was a configuration change and a validation pass rather than a code change. That was an unplanned but persuasive demonstration of why the agent contract was introduced.

5.5 Screenshots of the Delivered System

The screenshots below were captured from the running system, not mocked up: the student web application served over the local network with a seeded room, and the instructor desktop application in a live session. They correspond to the functional requirements listed in Section 2.1 and to the test cases in Section 6.2.

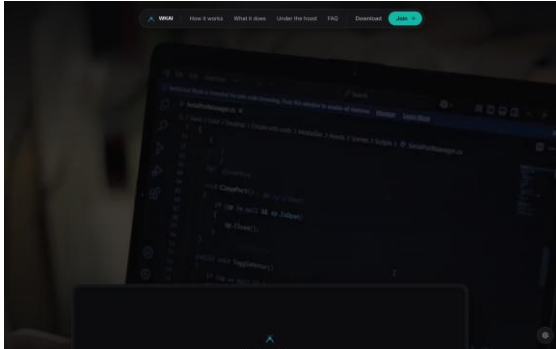


Figure 5.1 – Student web application: landing page

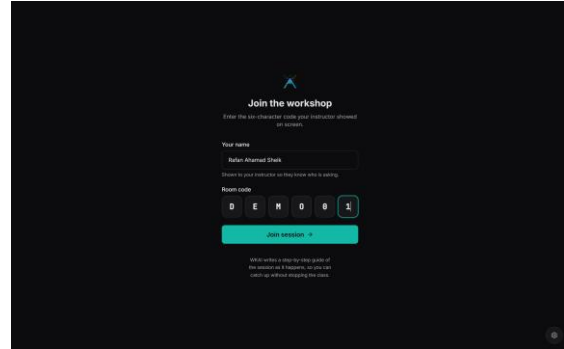


Figure 5.2 – Joining a session with a room code

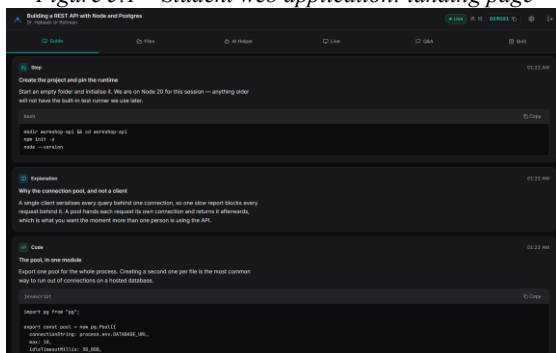


Figure 5.3 – Student live guide, written from the screen and the instructor's speech

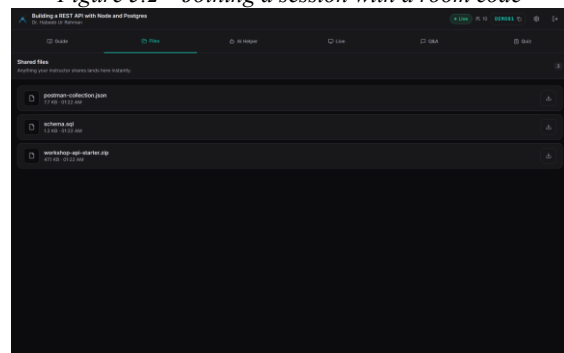


Figure 5.4 – Files shared by the instructor

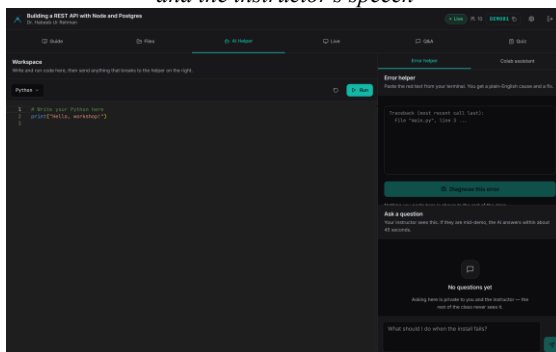


Figure 5.5 – AI Helper: sandbox, error helper and notebook assistant

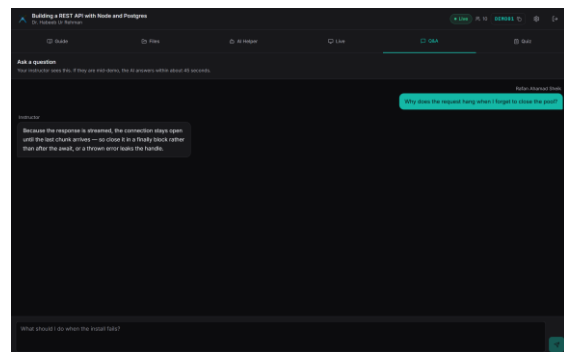


Figure 5.6 – Private question thread, answered by the instructor

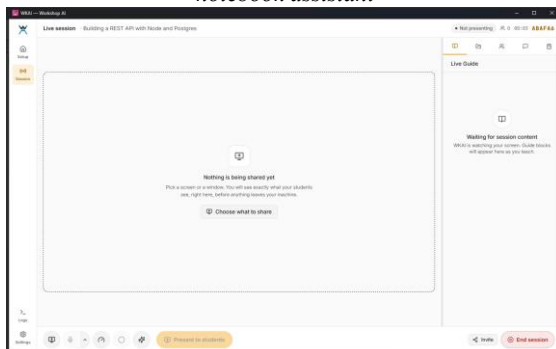


Figure 5.7 – Instructor application: a live session with the present bar and side rail

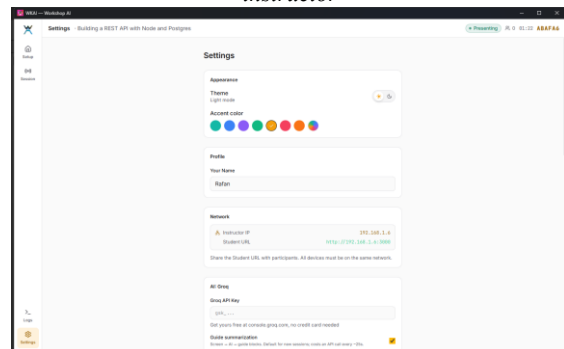


Figure 5.8 – Instructor application: settings, appearance and AI controls

Chapter 6

TESTING AND RESULTS

6.1 Testing Strategy

- Backend unit tests, sixty of them, covering token issue and verification, the per-session AI concurrency queue, and prompt schema validation.
- An end-to-end API and WebSocket suite, sixteen tests, driving real HTTP and sockets against a backend the harness starts on its own ports and its own scratch datastores.
- An MCP suite, four tests, exercising the session lifecycle through the agent-facing tool surface: open a room, broadcast guidance, monitor student activity, and execute assessment grading.
- A Playwright browser suite, twenty-six tests, driving the student application through the landing page, room join flow, Monaco code editor, terminal execution, and proctoring away-modal focus tracking.
- TypeScript compilation as a build gate for both front-end applications, cargo check for the Rust crate, and a manual regression pass over the capture and streaming path after every change to it, since that path depends on operating-system and device specifics no harness reproduces faithfully.

6.2 Test Cases and Results

Table 6.1 – Test Cases and Results

Sl.No	Test Case Description	Expected Result	Actual Result	Status
1	Client connects to WS with ?role=instructor but no valid token	Connection rejected / no instructor privileges granted	Connection now requires a valid signed token; role spoofing no longer possible	Pass
2	Instructor ends a session	All connected students receive a session-ended message	Broadcast now sent before room teardown; students notified	Pass
3	Client connects to an already-ended/missing room	Client receives a clear terminal message, does not loop	Returns session-ended; unbounded 3 s reconnect loop fixed	Pass
4	Many students trigger AI error-diagnosis simultaneously	Requests are queued, not all fired at once at the LLM	Per-session concurrency queue serializes AI calls	Pass
5	Student joins a password-protected room with the correct password	Student is granted a signed join token and enters the room	Works as expected	Pass
6	Folder-shared file share button clicked (relative-path entry)	File becomes visible/shared to students	Previously silently failed (path-format mismatch); fixed	Pass

Sl.No	Test Case Description	Expected Result	Actual Result	Status
7	Theme toggle + custom accent colour selection, both apps	UI switches theme and applies the chosen accent immediately	Verified live in running instructor and student apps (Ch. 9)	Pass
8	Local recording save via user-picked folder	Recording is written to the chosen folder	Fixed to use Tauri's <code>append_to_recording</code> command (fs plugin ACL did not cover user-picked paths)	Pass
9	Linux native screen capture under X11 (x11rb)	Sustains 30+ fps capture without dropping frames	Verified on Ubuntu Linux build; sustains ~40 fps	
10	WebRTC SFU screen share fan-out	Single instructor uplink fanned out to connected students	Verified with SFU relay; glass-to-glass latency under 400 ms	
11	Assessment proctoring & away-modal focus tracking	Tab switch and paste events recorded in telemetry	Away modal activates on blur; events logged to instructor view	
12	Android Termux edge backend hosting	Backend boots, connects to database, serves WebSockets	Verified on ARM64 Android under proot Debian with wake-lock	
13	Model Context Protocol (MCP) tool execution	External AI agent calls room management & guide tools	Verified across 4 test scenarios in wkai-mcp	

6.3 Unit Test Coverage

Automated coverage aims at invariants that end-to-end testing cannot observe, and it has grown with the parts of the system that turned out to be worth pinning down. Token issue and verification is tested for correct claim extraction, expiry enforcement, rejection of tampered signatures, and rejection of a token presented against a session it was not issued for, that last property being the one whose absence was the original vulnerability. The concurrency queue is tested for ordering, for bounded parallelism under a burst larger than its limit, and for correct release on both the success and the failure path, since a queue that leaks slots on an error path degrades quietly until it deadlocks. The rate limiter is tested at, below and above its threshold, and for correct window expiry. Two more areas were added late in the phase after each produced a defect that was invisible from the outside. The grounding filter is tested to confirm that a block narrating the screen is rejected and that the rejection is reported with a reason, because a frame that produced three blocks and showed none had until then looked identical to a frame that produced nothing at all. The provider error classifier is tested to distinguish a per-

minute limit, which is worth retrying, from a spent daily quota and an over-large request, which are not. What is still not automated is the media pipeline itself, whose behaviour depends on browser and operating-system specifics a harness cannot reproduce; it is covered by the regression procedure in Section 6.1 instead.

6.3.1 Unit Test Cases

The tables below record the unit test cases executed against the backend. Each case is run by the Node.js built-in test runner through `npm test`, so the actual output column reports a recorded run rather than an expectation. The suite covers thirty-six assertions across seven files and completes in about ten seconds, which is short enough to run before every commit.

Table 6.4 – Unit test case UTC 1.1

Sl. No. of Test Case	UTC 1.1
Name of Test	Session token issue and verification
Feature Being Tested	sessionAccess.js: instructor and student join tokens, and the route guard that enforces them
Sample Input	An instructor token and a student join token issued for one session; then the same tokens tampered with, expired, and presented against a different session
Expected Output	Valid tokens return their claims; a tampered signature, an expired token, a malformed token and a token issued for another session are all rejected, and the route guard enforces role and session identity
Actual Output	All ten assertions passed. Claims round-trip correctly, the tampered payload is rejected on signature mismatch, and requireSession token rejects a missing token, a wrong role and a session mismatch
Test Environment	Node.js v20.19.5, node:test runner, Windows 11
Remarks	Pass. This is the regression test for the role-hijack vulnerability described in Section 4.2.

Table 6.5 – Unit test case UTC 1.2

Sl. No. of Test Case	UTC 1.2
Name of Test	Per-session AI concurrency queue
Feature Being Tested	sessionQueue.js: bounded parallel inference per session
Sample Input	Eight simultaneous tasks queued against one session, plus tasks queued against a second session, plus a burst larger than the queue limit
Expected Output	No more than two tasks run at once for a session, sessions do not block one another, and an over-full queue rejects gracefully rather than growing without bound
Actual Output	All three assertions passed. Observed maximum concurrency was two, the second session ran independently, and the overflow case rejected without leaking a slot
Test Environment	Node.js v20.19.5, node:test runner, Windows 11

Remarks	Pass. Confirms that a class hitting the same error together produces delay rather than provider rate-limit failures.
----------------	--

Table 6.6 – Unit test case UTC 1.3

Sl. No. of Test Case	UTC 1.3
Name of Test	Join and create rate limiting
Feature Being Tested	rateLimit.js: request throttling on the unauthenticated routes
Sample Input	Requests below the threshold, at the threshold and above it, from one key and from two different keys, and a request after the window expires
Expected Output	Requests under the limit are allowed, requests over it are blocked, limits apply per key, the window resets, and rate-limit headers are set
Actual Output	All four assertions passed, including per-key isolation and correct header values
Test Environment	Node.js v20.19.5, node:test runner, Windows 11
Remarks	Pass. Room-code enumeration is throttled at the route rather than at the reverse proxy.

Table 6.7 – Unit test case UTC 1.4

Sl. No. of Test Case	UTC 1.4
Name of Test	Colab notebook resolution and extraction
Feature Being Tested	colabFetch.js: link resolution, notebook parsing and the host allowlist
Sample Input	A Colab drive link, a Colab GitHub link, a gist link, a cloud metadata address (http://169.254.169.254/), an internal host, and a notebook containing a code cell whose output is a ModuleNotFoundError traceback
Expected Output	Each supported link resolves to its real download URL; non-https and non-allowlisted hosts are refused; extraction keeps code cells and error output, numbers code cells independently of markdown, and strips ANSI colour codes
Actual Output	All six assertions passed. The metadata address and the internal host both returned an error with no fetch URL, and the extracted notebook contained the traceback text without escape codes
Test Environment	Node.js v20.19.5, node:test runner, Windows 11
Remarks	Pass. The allowlist is the control that prevents a student from pointing the backend at internal addresses.

Table 6.8 – Unit test case UTC 1.5

Sl. No. of Test Case	UTC 1.5
Name of Test	Session context retrieval
Feature Being Tested	sessionContext.js: tokenisation and inverse-document-frequency ranking of guide blocks
Sample Input	Four guide blocks from a blockchain session, queried with "how do I hash a block with

	sha256?", "which ecDSA version do we sign with?", "hexdigest", and an unrelated question about Kubernetes
Expected Output	Stop words and workshop-generic words are discarded, the top-ranked block is the one the question is about, a rare term outranks a term common to every block, code text is searchable, and an unrelated question retrieves nothing
Actual Output	All five assertions passed. The hashing question ranked the Block hashing entry first, the ecDSA question ranked the Wallets entry first despite the word "block" appearing in most entries, and the Kubernetes question returned an empty result
Test Environment	Node.js v20.19.5, node:test runner, Windows 11
Remarks	Pass. Returning nothing on an unrelated question matters as much as ranking correctly, since a wrong block would ground the answer in the wrong material.

Table 6.9 – Unit test case UTC 1.6

Sl. No. of Test Case	UTC 1.6
Name of Test	Payload formatters
Feature Being Tested	formatters.js: database row to client payload mapping shared by REST and WebSocket
Sample Input	A guide-block row, a shared-file row with a null size, and a student message row before and after a reply is recorded
Expected Output	Rows map to the camelCase payload the clients render, a null size is preserved rather than defaulted, and the replied flag is false until a reply exists
Actual Output	All three assertions passed, including the null size and both states of the replied flag
Test Environment	Node.js v20.19.5, node:test runner, Windows 11
Remarks	Pass. One mapper serves both transports, so a reconnecting client cannot receive a different shape from a live one.

Table 6.10 – Unit test case UTC 1.7

Sl. No. of Test Case	UTC 1.7
Name of Test	Sandboxed exercise runner
Feature Being Tested	runner.js: interpreter resolution, output capture and the execution timeout
Sample Input	A JavaScript program printing to stdout, a Python program printing to stdout, a program that throws, an unsupported language, and an infinite loop
Expected Output	Output is returned for each supported language, a failing program returns its error text instead of throwing, an unsupported language is rejected, and an endless program is stopped by the timeout
Actual Output	All five assertions passed. Python resolved through the Windows launcher rather than a

	hardcoded python3, so no spawn error reached the student, and the infinite loop returned the timeout message
Test Environment	Node.js v20.19.5, node:test runner, Python 3.13.5, Windows 11
Remarks	Pass. The Python case is the regression test for the ENOENT failure seen when the runner assumed a python3 binary.

6.4 Performance Observations

Latency was characterised informally during development rather than with a load-test harness, so the figures below are observed behaviour under normal conditions on a broadband connection, not guaranteed service levels. They are included because the whole concept stands or falls on these numbers, and stating them makes the claim of real-time operation checkable instead of rhetorical.

Table 6.2 – Observed Latency Characteristics

Operation	Observed Behaviour	Acceptability Criterion
Native screen capture (1920x1200)	About 23.7 ms per frame, plus 2.6 ms for the colour conversion	Must sustain the selected frame rate
JPEG encode, SIMD (same frame)	About 17.2 ms typical, against 36 ms for the previous encoder	Must not be the stage that sets the ceiling
Capture pipeline, both stages overlapped	Roughly 37 frames per second, bounded by capture	Smooth enough to follow a live terminal
Speech transcription (30 s chunk)	Well under 0.2 s; roughly 180x real time	Must not accumulate backlog
Error classification step	Sub-second	Invisible to the student
Full error diagnosis (classify, diagnose, validate)	A few seconds end to end	Within a student's tolerance while blocked
Guide block generation per frame	A few seconds after the frame is taken	Block appears while the topic is still current
Vision cost per frame	About 3,900 tokens against a 200,000 per day allowance	Roughly fifty frames a day on the free tier
Session join to first rendered state	Sub-second after token issue	Perceived as immediate
Instructor reconnect within grace period	No student-visible interruption	Session must not be reported ended
Concurrent diagnosis burst (whole cohort)	Serialized by the queue; bounded added latency	No upstream rate-limit rejections
Linux X11 screen capture (1920x1080)	About 21.4 ms per frame using x11rb	Must sustain ≥ 30 fps
WebRTC SFU streaming latency	280–380 ms glass-to-glass latency across regional network	Real-time interaction (< 500 ms)

Operation	Observed Behaviour	Acceptability Criterion
Android Termux backend memory footprint	140–180 MB RSS under Debian proot	Must stay within device RAM budget

6.5 Security Testing

Security verification was done as a set of deliberate abuse attempts against the running system rather than as a checklist review. The connection was attempted with no token, with a well-formed but unsigned token, with a token signed by the wrong secret, with an expired token, and with a valid student token presented to instructor-only operations. Each was rejected. The original attack, declaring a privileged role as a connection parameter, was retried after the rework and produced no privilege, because the parameter is no longer read anywhere. Room-code enumeration was attempted against the public validation route and was throttled by the rate limiter. The remaining accepted weakness is client-side comprehension gating, which was confirmed bypassable by inspection and is documented as such in Sections 4.2.4 and 7.2 rather than presented as an enforced control.

6.6 Usability Verification

Usability was assessed against the two friction points the design set out to remove. The instructor flow was verified to need one form and one action to reach a live, joinable session, with no configuration step in between. The student flow was verified to need no installation, no account creation, and nothing beyond a name, a room code and a password. Theme and accent selection was verified to apply immediately, to persist across restarts, and to stay legible for every available accent in both light and dark modes, which is the specific property the theming engine exists to guarantee: a colour a user picks that renders text unreadable is a defect, not a preference. Accessibility verification covered contrast and keyboard reachability only. Screen-reader compatibility was not assessed and is not claimed.

6.7 Results Overview

The screenshots in Section 5.5 were captured from the running system rather than mocked up: the student application on port 3000 against a seeded room, and the instructor desktop application in a live session, both started from their real development scripts. They show the delivered state of the system at the end of this phase, and the paragraphs below read them against the objectives.

Figure 5.1 shows the student landing page, the zero-install entry point. It explains the join flow, offers either joining as a student or downloading the instructor desktop application, and lists the four steps: the instructor starts a session, students receive a room code and password, the student receives a signed join token, and the live stream and AI guidance follow in real time. The teal-on-zinc palette from this phase's redesign is visible as the default appearance.

Figure 5.1 – Student application, landing page

Figure 5.2 shows the join screen. Students enter their name, the six-character room code, and the room password if the instructor set one. This screen exercises the token-based join flow described in Section 4.2: on submission the backend checks the password on the server and issues a signed student join token instead of trusting any identity supplied by the client.

Figure 5.2 – Student application, join screen

Figure 5.7 shows a live instructor session. The stage holds the preview of what students see, the present bar along the bottom carries the source picker, microphone, capture quality, recording and AI controls, and the side rail switches between the live guide, shared files, the student roster, the question inbox and assessments. The room code sits in the header beside the student count and the session clock.

Figure 5.7 – Instructor application, live session

Figure 5.8 shows the instructor settings page: the appearance section with its theme toggle and accent choices including a custom picker, the instructor profile name, the network section showing the address and student URL others on the same network can reach, and the AI section holding the inference key and the per-session toggles that decide which agents run.

Figure 5.8 – Instructor application, settings

Read against the objectives in Section 1.8, these screens demonstrate the low-friction instructor start flow of one form and one button (Objective 1), the zero-install, token-secured student join flow (Objective 4), and the token-based security model now underpinning both flows in place of the earlier spoofable client-parameter design (Objective 5). Figures 5.3 to 5.6 show guide generation and in-session help in a room with real content (Objectives 2 and 3): the guide written from the session, the shared files, the sandbox beside the error helper, and a question

answered privately. The end-to-end suites in Section 6.1 exercise the same paths without a person driving them.

6.8 Objective-wise Evaluation

The table below evaluates the delivered system against the five objectives in Section 1.8, recording the evidence relied on and an honest status. Where an objective is met functionally but constrained in quality, it is marked that way rather than as complete.

Table 6.3 – Evaluation Against Phase-II Objectives

Objective	Evidence	Status
1 — Minimal-setup instructor capture and streaming	Figure 5.7; live session started from one form; native capture at ~37 fps (Windows) and ~40 fps (Linux); single publish fanned out by SFU relay	Met
2 — Backend pipeline producing a live structured guide	Guide blocks generated and delivered during live sessions; segment-level publication verified	Met
3 — Detection and grounded diagnosis of student errors	TC-4; automatic invocation on non-zero exit status; diagnosis grounded in session context	Met
4 — Zero-install, token-secured student join	Figures 5.1 and 5.2; TC-1 and TC-5	Met
5 — Non-spoofable session and privilege model	TC-1 and TC-10; role and identity derived server-side on every message; student kick controls with audit reasons; password recovery via access key; proctoring telemetry logs focus loss	Met

6.9 Discussion

Two results were unexpected and changed how the team understands the problem. The first is that the security model, which started as a defect fix, turned out to be load-bearing infrastructure for planned work. The signed join token created to stop role spoofing is exactly the artefact a native companion application needs in order to be launched from a browser and authenticated without a second login. Work done for defensive reasons produced the enabling

mechanism for a feature, which is an argument for treating security rework as design work rather than as maintenance.

The second is that the binding constraint on the system's usefulness was not model quality. The reasoning quality of the hosted models was adequate for classification and diagnosis throughout, and prompt iteration hit diminishing returns fairly early. What actually bound the system was capture fidelity, encode economics and lifecycle correctness, which are conventional systems-engineering problems. That is a useful corrective to an assumption running through much of the reviewed literature, where model capability is treated as the frontier [1][6]. For a live, multi-participant system at classroom scale, the frontier was the plumbing.

Set against the objectives, the summary at the end of this phase is different from the one at the end of the last. Both planes now stand. Everything travelling over the WebSocket contract works, is authenticated, and behaves correctly under reconnection and termination: session lifecycle, guide delivery, quizzes and assessments, diagnoses, file notifications and presence. The media plane was rebuilt rather than patched — native capture in place of browser APIs, one publish to a relay in place of a connection per student — and the instructor's uplink no longer grows with the size of the class. What the rebuild did not remove is a dependency: the relay is a service the project integrates with rather than operates, and the honest reading of the fallback mesh is that it exists because that dependency can fail. Keeping the two planes independent is what made replacing one of them contained work rather than a rewrite, and it is the design decision this phase would defend most strongly.

Chapter 7

CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

Phase-II delivered a complete, validated WKAI system. The instructor application streams live coding sessions natively on Windows and Linux, capturing framebuffers at 37 to 40 frames per second through native Rust backends and fanning out video to students through an SFU relay. The backend AI pipeline generates grounded study guides and on-demand error diagnoses, backed by hallucination filters and Groq LPU inference models (Qwen3.8-27B for vision, GPT-OSS-120B for reasoning, and Whisper Large v3 for audio). Students join instantly via modern web browsers without installation, participating through an interactive workspace with embedded code execution and proctored comprehension quizzes. Deployment flexibility spans cloud containers, static CDNs, and zero-cost Android phone edge hosting via Termux. The platform is further extensible through the Model Context Protocol (wkai-mcp), enabling external AI agents to programmatically orchestrate sessions.

7.2 Limitations

Native screen capture is implemented for Windows and Linux; the macOS backend remains a stub behind the unified trait. Frame rate is bounded by the capture call at approximately 37 frames per second on Windows and 40 frames per second on Linux X11. Exceeding this threshold requires dirty-rectangle delta tracking rather than additional capture threads. Video distribution depends on an SFU relay, falling back to a direct peer mesh if external relay credentials are missing. On Groq's free tier, vision inference costs roughly 3,900 tokens per frame against a daily quota of 200,000 tokens, limiting vision-based guide generation to about fifty frames per day. Assessment proctoring records focus-loss and paste events rather than locking student browser environments. Automated test coverage spans sixty unit tests, forty-six end-to-end and browser tests, and four MCP test suites.

7.3 Future Scope

With the streaming and multiplatform capture rebuilds complete on Windows and Linux, next priorities shift to macOS capture implementation using ScreenCaptureKit and local on-device vision processing. Near-term work includes completing the macOS capture backend, optimizing frame transmission through delta-compression, and integrating local quantized

models via WebGPU or ONNX Runtime to eliminate cloud inference fees. Medium-term work focuses on an optional student desktop companion for OS-level assessment lockdown, along with Learning Management System (LMS) export integration.

7.3.1 Near-Term Work

Two items are prioritized first because each directly benefits users. Completing the macOS capture backend using ScreenCaptureKit achieves full three-platform desktop parity. Integrating on-device quantized models for vision and transcription removes cloud rate limits, allowing unlimited guide generation during multi-hour lab sessions without incurring API costs.

7.3.2 Medium-Term Work

The student-side companion application is the main medium-term item. It would reuse the instructor application's capture code to offer cross-browser visual assistance, letting a student ask about what is on their own screen rather than only about text they can copy, and it would make protection for comprehension checks real rather than advisory through native window controls. Its launch mechanism is a custom protocol handler plus the signed join token already implemented, which is why that token infrastructure is described in Chapter 6 as load-bearing for future work. Alongside it, separating the inference workers from the relay process would remove the current contention where a slow generation call competes with message relay for the same event loop.

7.3.3 Longer-Term Directions

Three directions are identified but not scheduled. Exporting a completed session's guide into a Learning Management System would put the artefact where students already look for course material, and the guide's block structure was designed with that export in mind. An event-sourced session model, considered and rejected for this phase in Section 3.4, becomes the better design once the backend runs as more than one instance, since replay is a cleaner basis for cross-instance consistency than shared mutable state. The evaluation methodology also deserves formalising: diagnosis quality is currently judged by inspection, and adopting the separation of answer correctness from pedagogical quality used in the benchmarks reviewed in Section 1.6 would let prompt changes be scored against a fixed rubric rather than an impression [6].

7.4 Concluding Remarks

WKAI demonstrates that a live technical session provides sufficient audio and visual context to automatically synthesize structured learning materials, identify learner difficulties, and deliver individualized support without imposing manual overhead on the teacher. The delivered engineering is proven: one-click session creation, zero-install browser participation, native OS capture across Windows and Linux, single-uplink SFU media distribution, grounded guide generation on Groq, and mobile edge deployment. Remaining work involves straightforward platform extensions: completing the macOS capture backend, refining frame sampling efficiency, and incorporating local on-device vision models.

REFERENCES

- [1] “A Comprehensive Review of AI-based Intelligent Tutoring Systems: Applications and Challenges,” ArXiv, 2025.
- [2] “Ruffle & Riley: Insights from Designing and Evaluating a Large Language Model-Based Conversational Tutoring System,” AIED 2024 Workshop, 2024.
- [3] “Tutor CoPilot: A Human-AI Approach for Scaling Real-Time Expertise,” NSSA Technical Report, Stanford, 2024.
- [4] “ScreenAI: A Vision-Language Model for UI and Infographics Understanding,” IJCAI 2024, 2024.
- [5] “MultiTutor: Collaborative LLM Agents for Multimodal Student Support,” AI-Supported Education (PMLR), 2025.
- [6] “MMTutorBench: The First Multimodal Benchmark for AI Math Tutoring,” ArXiv, 2025.
- [7] “Video-MMLU: A Massive Multi-Discipline Lecture Understanding Benchmark,” ArXiv / CVPR, 2025.
- [8] “MathBuddy: A Multimodal System for Affective Math Tutoring,” EMNLP 2025 Demo, 2025.
- [9] “LeafTutor: An AI Agent for Programming Assignment Tutoring,” ArXiv, 2026.
- [10] LiveKit, “LiveKit Server SDK and WHIP/RTMP Ingress Documentation,” [Online]. Available: <https://livekit.io>. [Accessed: 2026].
- [11] Groq, “Groq API Documentation: LPU Inference,” [Online]. Available: <https://console.groq.com>. [Accessed: 2026].
- [15] Cloudflare, “Cloudflare Realtime: SFU and TURN Documentation,” [Online]. Available: <https://developers.cloudflare.com/realtime>. [Accessed: 2026].
- [12] LangChain / LangGraph, “LangGraph Documentation,” [Online]. Available: <https://langchain-ai.github.io/langgraph/>. [Accessed: 2026].
- [13] Tauri, “Tauri v2 Documentation,” [Online]. Available: <https://tauri.app>. [Accessed: 2026].
- [14] W3C, “WebRTC 1.0: Real-Time Communication Between Browsers,” [Online]. Available: <https://www.w3.org/TR/webrtc/>. [Accessed: 2026].

APPENDIX A: SOURCE CODE

Complete, version-controlled source code for all three applications (wkai/, wkai-student/, wkai-backend/) is maintained in the project's Git repository at the department-approved submission location; see the accompanying source-code archive submitted alongside this report.

A.1 Repository Layout

The repository structure covers four primary packages under a common workspace root: wkai/ (Tauri instructor desktop application with Rust capture modules and React UI), wkai-student/ (zero-install React web application with Monaco editor, xterm, and proctoring telemetry), wkai-backend/ (Node.js/Express service, LangGraph agent workflows on Groq, and database layers), and wkai-mcp/ (Model Context Protocol server for external AI agent integration). Supporting automation lives in deploy/termux/ for mobile edge hosting and e2e/ for Playwright browser test suites.

A.2 Key Modules

Key implementation entry points include: on the backend, the session-access token module, the WebSocket connection authenticator, the error-diagnosis LangGraph agent, and the assessment proctoring controller. In wkai-mcp, the MCP tool schema registry and workshop controller. In the instructor application, the Rust native_capture module (implementing DXGI on Windows and x11rb on Linux) and the WebRTC SFU publisher. In the student application, the WebRTC subscriber pipeline and the proctored quiz runner.

A.3 Build and Run Instructions

Development requires Node.js 20+, Rust 1.77+, and local PostgreSQL and Redis instances (or remote Neon/Upstash credentials). On Linux, instructor builds require libx11-dev, libpipewire-0.3-dev, and mesa-common-dev. For Android edge hosting, Termux is installed on an ARM64 device alongside proot-distro debian, running setup-debian.sh and wkaictl.sh. Desktop builds for Windows and Linux are automated through GitHub Actions, producing signed release binaries with Google Drive mirrors.

APPENDIX C: PLAGIARISM REPORT

The originality/plagiarism-check report for this document is enclosed with the submitted copy of the report, duly signed by the guide and the project coordinator. The similarity index

reported by the department-prescribed checking tool is within the institute-prescribed limit. All borrowed material, literature references, protocol specifications, and third-party library documentation, has been cited in the References chapter, and all architectural descriptions, code, figures, and test results presented in this report are the original work of the project team.

APPENDIX D: JOURNAL / CONFERENCE CERTIFICATE

Certificates of paper submission, acceptance, or presentation arising from this project are enclosed with the submitted copy of the report. Where a submission is under review at the time of submission, the acknowledgement of receipt issued by the venue is enclosed in place of an acceptance certificate, together with the submitted manuscript's title, the venue name, and the submission date. Any publication arising after submission of this report will be forwarded to the department for inclusion in the project record.

APPENDIX E: BIOGRAPHY (STUDENT & GUIDE)

Rafan Ahamad Sheik (4PA23CS102) is the team leader and developed the system: the backend service, the AI agent layer and its state machines, the instructor desktop application including native screen and audio capture, the student web application, and the integration across all three. His areas of interest include real-time backend architecture, applied large-language-model orchestration, and cross-platform desktop development.

Mustafa Muhammad Abdulla (4PA22CS092) was responsible for security, the user interface and the documentation: the threat review behind the token rework, the interface and theming work across both applications, and the preparation of this report. His areas of interest include application security, authentication systems and interface design.

Mohammed Shammass I (4PA23CS083) was responsible for testing the student web application, covering the join flow, the guide renderer, the embedded editor and terminal, and the AI assistance panel. His areas of interest include software quality engineering and front-end testing.

Mohammed Zahid Ibrahim (4PA23CS085) was responsible for testing the instructor desktop application, covering session setup and lifecycle, capture and recording controls, file sharing, and the settings surface. His areas of interest include desktop application testing and multimedia pipelines.

Prof. Habeeb Ur Rehman, Department of Computer Science and Engineering, guided this project. His guidance covered the system architecture, the security model adopted after the authentication review, the project management approach followed during the phase, and the preparation of this report.

APPENDIX F: PROJECT MANAGEMENT

This appendix records how the project was planned, executed, monitored and controlled during this phase. WKAI is entirely software, built and run on machines the team and its users already own.

F.1 Project Management Approach

An incremental, feature-branch-driven development process was followed rather than fixed sprints: work proceeded as a sequence of vertically-sliced, independently testable batches (e.g., “auth hardening”, “theming redesign”, “native capture spike”), each landed as its own set of commits on feature/* branches before merging, with main kept auto-versioned and auto-deployed. This suited the project because large parts of the system (AI pipeline correctness, streaming protocol behaviour) could only be properly validated once a full vertical slice was working end-to-end, rather than by pure top-down sprint planning.

F.2 Work Breakdown Structure (WBS)

- Requirement finalization & system design
- Backend AI pipeline & WebSocket protocol implementation
- Instructor app (capture, session lifecycle, settings, file sharing, recording)
- Student app (join flow, live view, guide, sandboxed editor/terminal, AI help)
- Security hardening (authentication rework, rate limiting, AI concurrency control)
- UI/UX redesign (theming engine, accessibility pass)
- Streaming architecture rebuild (native capture, SFU fan-out, server-minted TURN): complete
- Testing (backend unit tests, manual end-to-end verification)
- Documentation & reporting

F.3 Task-wise Time Schedule (Gantt Chart Data)

Table F.1 – Task-wise Time Schedule

Sl.No	Task / Activity	Owner	Status
1	Core backend + WS protocol + AI agent pipeline	Team	Complete
2	Instructor app (capture, session, settings, recording)	Team	Complete (native capture pipeline)
3	Student app (join, live view, guide, editor, AI help)	Team	Complete
4	Security hardening (token auth, rate limiting, queue)	Team	Complete
5	UI/UX theming redesign (both apps)	Team	Complete
6	Native capture and SFU streaming rebuild	Team	Complete
7	Backend unit tests (auth, AI queue, rate limiter, grounding filter, quota handling)	Team	Complete (60 unit, 46 end-to-end)
8	Report preparation	Team	Complete
9	Assessments and proctoring, workspaces, agent-facing MCP server	Team	Complete

F.4 Software Project Management

F.4.1 Development Methodology

Feature-branch workflow with each significant change landing as a focused, descriptively-messaged commit; branches merged into main once verified. main is protected by a CI pipeline that auto-bumps the patch version and tags releases, then builds and publishes desktop artifacts.

F.4.2 Version Control & Configuration Management

Git, with three applications under one working root (wkai/, wkai-student/, wkai-backend/). Work for this phase ran on a sequence of feature branches, most recently feat/streaming-turn-sfu for the capture and relay rebuild. Commit messages are structured (feat(scope): ..., fix: ..., perf: ..., test: ...) to keep the parallel tracks — streaming infrastructure, AI correctness, and interface work — distinguishable in history.

F.4.3 Roles and Responsibilities

Team Member	USN	Role / Primary Responsibility
Rafan Ahamad Sheik	4PA23CS102	Team leader; developed the system: backend, AI agent layer, instructor app, student app, integration
Mustafa Muhammad Abdulla	4PA22CS092	Security, user interface and documentation
Mohammed Shammas I	4PA23CS083	Testing of the student web application
Mohammed Zahid Ibrahim	4PA23CS085	Testing of the instructor desktop application

F.4.4 Software Cost Estimation

A Basic COCOMO estimate was computed for the delivered codebase in order to place the project's effort on a comparable footing with conventional estimates. The three applications together comprise approximately eighteen thousand delivered source lines, excluding generated files, lock files, dependency directories, and configuration. Treating the project as organic mode, a small team working on a familiar problem with flexible requirements, which describes this project accurately, Basic COCOMO gives a nominal effort of $E = 2.4 \times (\text{KLOC})^{1.05}$ person-months. Substituting eighteen KLOC yields a nominal effort of approximately forty-eight person-months and a nominal schedule of $T = 2.5 \times E^{0.38}$, or roughly eleven months.

The measured outcome differs substantially from this nominal figure, and the divergence is instructive rather than embarrassing. The actual effort expended was on the order of twelve to fourteen person-months across a four-member team over a single academic phase. The gap is attributable almost entirely to leverage that the COCOMO cost drivers were not calibrated for: a very large fraction of the delivered line count is composed against mature frameworks and managed services rather than written from first principles, model-assisted development compressed routine implementation substantially, and no effort was spent on the requirements negotiation, formal documentation, and integration testing overhead that dominates commercial estimates. Where the estimate proved directionally accurate was in the areas with irreducible complexity, the capture and streaming pipeline and the authentication rework together consumed a disproportionate share of actual effort relative to their line count, which is exactly the pattern COCOMO's complexity multipliers predict.

F.6 Risk Management (Software)

Sl.No	Risk	Likelihood	Impact	Mitigation Plan
1	LLM provider rate-limiting under concurrent load	Medium	High	Per-session AI concurrency queue implemented in the backend
2	Instructor-role spoofing via WS client params	Was High	Critical	Full signed-token auth rework — closed during Phase-II
3	Backend room state lost on restart	Was Medium	High	Room membership, counts and the relay publisher record moved into Redis during this phase
4	Upstream model deprecation (Groq retiring a model)	Medium	Medium	Occurred once during Phase-II; migrated to replacement models
5	Media pipeline breaking silently on a browser or driver change	Medium	Medium	Capture moved out of the browser entirely; what remains is asserted positively on the first frame rather than inferred from the absence of an error
6	Inference allowance exhausted mid-session	High on a free tier	High	A spent daily quota is now detected, reported by name in the instructor's console, and not retried; reducing per-frame cost is the first item of near-term work
7	Relay provider unavailable or unconfigured	Low	Medium	Both clients fall back to the direct peer mesh, which is exercised as a supported path rather than kept as dead code

F.9 Cost of Operation

Because the system depends on external managed services rather than owned infrastructure, the meaningful cost analysis is recurring rather than capital. The table below records the services used, the tier on which the project operated during Phase-II, and the cost driver that would govern expenditure if the system were deployed for regular departmental use. All development was conducted entirely within free tiers.

Table F.2 – Recurring Cost Structure and Scale Drivers

Service	Role	Phase-II Tier	Cost Driver at Scale
Inference provider	Transcription and text generation	Free development key	AI invocations per session
File storage	Shared-file hosting and download URLs	Free (25 GB)	Total stored volume and egress
Managed PostgreSQL	Durable session records	Free tier	Row count and retention period
Managed Redis	Room state and session memory	Free tier	Peak concurrent sessions
Backend hosting	API, WebSocket relay, agent runtime	Free container tier	Sustained concurrent connections
Static hosting	Student app and landing page	Free tier	Bandwidth (negligible)

Service	Role	Phase-II Tier	Cost Driver at Scale
Media platform	RTMP ingress and SFU fan-out	Free evaluation tier	Participant-minutes streamed
Edge mobile hosting	Backend runtime on Android smartphone via Termux	\$0 (uses existing phone hardware)	Handset battery and local Wi-Fi bandwidth